

Reinforcement Learning

Interview-focused notes for ML placements, AI engineer roles, research internships, and projects. The guide progresses from the agent-environment interface to modern topics such as PPO, offline RL, RLHF, and MuZero. Symbols used throughout:

Symbol	Meaning
S_t, A_t, R_{t+1}	State, action, and next reward at time t
$\pi(a \mid s)$	Policy: probability of action a in state s
$\gamma \in [0, 1]$	Discount factor
$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$	Discounted return
$V^{\pi}(s), Q^{\pi}(s, a)$	State-value and action-value under policy π

Agent

1. Overview

An agent is the decision-making component in reinforcement learning (RL). It observes information from an environment, selects actions, receives rewards, and improves its behavior to maximize expected long-term return. Agents appear in robots, recommender systems, game-playing systems, resource schedulers, autonomous vehicles, and language-model alignment pipelines.

Unlike a supervised predictor that maps labeled inputs to outputs once, an RL agent affects the data it will see next. This feedback loop makes credit assignment, exploration, safety, and non-stationarity central concerns.

2. Intuition

Imagine a learner playing an unfamiliar video game. The screen is its observation, joystick moves are actions, score changes are rewards, and the game is the environment. The player gradually discovers which sequences of moves produce high scores. A strong agent is not merely reactive: it considers delayed effects, such as collecting a key now to open a door later.

3. Prerequisites

- Probability, conditional probability, expectation, and sampling
- Vectors, matrices, derivatives, gradients, and optimization
- Python and NumPy; PyTorch for neural agents
- Basic supervised learning and neural networks
- States, actions, rewards, policies, returns, and MDPs

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Decision rule	Mechanism choosing actions	Defines behavior	Move toward food	Agent versus policy
Internal state	Memory or learned representation	Handles partial observability	RNN remembers earlier frames	State versus observation
Objective	Quantity the agent optimizes	Specifies desired behavior	Expected discounted score	Reward versus return
Learning algorithm	Procedure updating behavior	Determines data efficiency and stability	Q-learning update	Model-free versus model-based
Experience	Transitions collected by interaction	Training data depends on policy	(s, a, r, s') tuple	On-policy versus off-policy

An agent can be reflex-based, value-based, policy-based, actor-critic, model-based, single-agent, or multi-agent. "Agent" describes the whole decision-maker; "policy" is only its action-selection rule.

5. Algorithm / Working Process

1. Initialize a policy, value estimates, model, or some combination.
2. Receive observation o_t ; convert it to an internal state or feature vector.
3. Select a_t , possibly using exploration.
4. Execute a_t in the environment.
5. Receive r_{t+1} , the next observation, and a termination signal.
6. Store or immediately learn from the transition.
7. Update parameters to increase expected future return.
8. Repeat across steps and episodes; evaluate without training noise.

Input is current information and possibly memory. Output is an action or action distribution. Training changes parameters; inference only computes the action.

6. Mathematical Foundation

The standard objective is

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T-1} \gamma^t R_{t+1} \right],$$

where trajectory $\tau = (s_0, a_0, r_1, \dots)$ is partly generated by the agent. A stochastic agent samples $A_t \sim \pi_\theta(\cdot \mid S_t)$; a deterministic agent uses $A_t = \mu_\theta(S_t)$.

The data distribution is policy-dependent:

$$d^\pi(s) = \Pr(S_t = s \mid A_t \sim \pi).$$

This is why changing the agent also changes its training examples. The agent seeks parameters $\theta^* = \arg \max_{\theta} J(\pi_{\theta})$, commonly using temporal-difference errors or policy gradients rather than differentiating through the real environment.

7. Practical Implementation

```
import numpy as np

class BanditAgent:
    """Epsilon-greedy agent for a stationary multi-armed bandit."""
    def __init__(self, n_actions, epsilon=0.1):
        self.epsilon = epsilon
        self.values = np.zeros(n_actions)
        self.counts = np.zeros(n_actions, dtype=int)

    def act(self):
        if np.random.random() < self.epsilon:
            return np.random.randint(len(self.values))
        return int(np.argmax(self.values))

    def learn(self, action, reward):
        self.counts[action] += 1
        step_size = 1.0 / self.counts[action]
        self.values[action] += step_size * (reward - self.values[action])

rng = np.random.default_rng(7)
true_means = np.array([0.2, 0.8, 0.5])
agent = BanditAgent(3)
for _ in range(5_000):
    action = agent.act()
    reward = rng.normal(true_means[action], 1.0)
    agent.learn(action, reward)

print("estimated values:", agent.values)
print("chosen best arm:", np.argmax(agent.values))
```

8. Code Explanation

The agent stores an estimated reward and visit count for each action. “act” balances random exploration with greedy exploitation. “learn” uses an incremental sample mean, so it needs no replay buffer. This is a context-free agent: there are actions and rewards but no state transitions.

9. Training / Evaluation

Separate learning behavior from evaluation behavior. During training, retain exploration; during evaluation, freeze parameters and use a deterministic or low-temperature policy. Report mean return over many seeds and episodes, standard deviation or confidence intervals, sample efficiency, success rate, and constraint violations. Training and evaluation environment wrappers must agree on observation and reward semantics.

10. Complexity and Cost

For a tabular agent, action selection is often $O(|A|)$ and storage may be $O(|S||A|)$. A neural agent's inference cost is one forward pass; training adds backward passes and potentially replay memory. Environment interaction often dominates wall-clock cost in robotics, while model training dominates in simulators and large-scale RLHF.

11. Common Use Cases

- Game agents and planning systems
- Robot control and navigation
- Recommenders, ranking, and ad allocation
- Traffic-signal, cloud-resource, and inventory control
- LLM post-training and preference optimization

12. Common Mistakes

- Calling the policy itself "the agent" and ignoring memory, learning, and preprocessing
- Optimizing immediate reward rather than return
- Evaluating with exploration still enabled
- Comparing agents using one random seed
- Letting training information leak through simulator state
- Ignoring unsafe exploratory actions

13. Edge Cases / Limitations

Sparse rewards can leave an agent without a useful learning signal. Partial observations may require memory. Distribution shift makes a deployed agent encounter unseen states. Reward misspecification can produce high measured reward but undesirable behavior. Physical interaction may be slow, expensive, or unsafe.

14. Variations

Variation	What changes	When to use	Importance
Value-based	Learns values, derives policy	Small/discrete actions	Placement core
Policy-based	Directly learns action distribution	Continuous/stochastic control	Placement and research
Actor-critic	Learns policy and critic	General deep RL	Very important
Model-based	Learns/uses dynamics	Interaction is costly	Projects/research
Hierarchical	Chooses goals and low-level actions	Long-horizon tasks	Advanced

15. Related Topics

An agent interacts with an **environment**, receives a **state/observation**, chooses an **action** using a **policy**, and learns from **reward**. Value functions estimate future return. In partially observable MDPs, recurrent networks or belief states supply memory.

16. Interview Questions

1. **What is an RL agent?** The entity that maps available information to actions and learns to maximize expected return.
2. **Agent versus policy?** The policy is the action rule; the agent may also contain memory, value functions, a model, and a learning procedure.
3. **Why is RL data non-IID?** Consecutive transitions are correlated and their distribution depends on the current policy.
4. **What does an agent optimize?** Expected cumulative, usually discounted, reward—not necessarily the next reward.
5. **Can an agent be deterministic?** Yes; $a = \mu(s)$, though exploration noise may be added during training.
6. **What is an episodic agent?** One whose interaction naturally resets after terminal states.
7. **Why use multiple random seeds?** RL outcomes have high variance from initialization, exploration, and environment randomness.
8. **What is credit assignment?** Determining which earlier actions caused a later reward.
9. **Why can a high-reward agent be unsafe?** The specified reward may omit real constraints or be exploitable.
10. **Training versus inference?** Training collects experience and updates parameters; inference freezes parameters and selects actions.

17. Practice Tasks

- Code an epsilon-greedy bandit agent.
- Compare optimistic initialization with random exploration.
- Plot average reward and optimal-action rate over time.
- Debug an agent accidentally evaluated with $\epsilon > 0$.
- Extend the agent to a non-stationary bandit using constant step size.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Smart traffic agent	Controls intersection phases	Python, Gymnasium, SUMO	SUMO traffic simulations	Sequential control and evaluation
Adaptive recommender	Chooses items from feedback	NumPy/PyTorch	MovieLens replay simulator	Connects RL and ranking
Warehouse navigator	Routes a robot through obstacles	PyTorch, Gymnasium	MiniGrid/custom grid	Demonstrates environment design

19. Quick Revision

- **Key idea:** the agent is the complete learner/decision-maker.
- **Main objective:** maximize $J(\pi) = \mathbb{E}[G_0]$.
- **When to use:** decisions change future outcomes and feedback arrives sequentially.

- **Metrics:** return, success rate, sample efficiency, safety violations.
- **Common trap:** equating immediate reward with the objective.
- **Interview one-liner:** "An agent closes the perception-action-learning loop with its environment."

20. Final Cheat Sheet

Item	Summary
Definition	Decision-maker that learns from interaction
Input/output	Observation or state → action
Main steps	Observe, act, receive reward, update
Hyperparameters	Exploration rate, discount, learning rate
Metrics	Episodic return, success, stability, safety
Pros/cons	Adapts to sequential feedback / can be unstable and data-hungry
Best uses	Long-term sequential decisions

Environment

1. Overview

The environment is everything external to the agent that receives actions and produces observations, rewards, and termination information. It may be a physical robot's world, a game engine, a user-response simulator, a market simulator, or a learned world model. Environment design determines what problem is actually being solved.

2. Intuition

In chess, the agent chooses a move; the board and opponent apply the rules, produce a new board, and eventually signal win, draw, or loss. The environment is both the rulebook and the process that advances the world.

3. Prerequisites

Probability, random variables, state transitions, MDPs, episodic versus continuing tasks, Python classes, and experiment reproducibility.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Transition dynamics	$P(s' s, a)$	Determines consequences	Slippery grid movement	Known versus unknown model
Reward function	$R(s, a, s')$	Defines learning signal	+1 at goal	Reward design risks

Subtopic	Meaning	Why it matters	Example	Interview angle
Observation space	What agent can perceive	May hide true state	Camera pixels	POMDP
Action space	Legal controls	Shapes algorithm choice	Discrete left/right	Discrete versus continuous
Reset/termination	Episode boundaries	Affects targets and metrics	Game over	Terminated versus truncated

5. Algorithm / Working Process

An environment exposes two central operations. “reset” initializes a trajectory and returns an observation. “step(action)” validates the action, advances dynamics, computes reward, and returns next observation, reward, “terminated”, “truncated”, and diagnostic information. The agent must not directly access hidden simulator variables unless they are explicitly part of the observation.

6. Mathematical Foundation

For an MDP, dynamics are

$$P(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

A stochastic environment samples from this distribution. A deterministic environment is a special case with $s' = f(s, a)$ and $r = g(s, a, s')$. The initial state follows $S_0 \sim \rho_0$. A time limit causes truncation, while an MDP terminal condition causes termination; bootstrapping commonly stops only on true termination.

7. Practical Implementation

```
class LineWorld:
    """Five cells; reach cell 4 for +1, maximum 10 steps."""
    def __init__(self):
        self.n = 5

    def reset(self):
        self.state, self.steps = 0, 0
        return self.state, {}

    def step(self, action):
        if action not in (0, 1):
            raise ValueError("action must be 0 (left) or 1 (right)")
        self.state = max(0, self.state - 1) if action == 0 else min(4, self.state
+ 1)
        self.steps += 1
        terminated = self.state == 4
        truncated = self.steps >= 10 and not terminated
        reward = 1.0 if terminated else -0.01
        return self.state, reward, terminated, truncated, {}

env = LineWorld()
```

```
obs, _ = env.reset()
done = False
while not done:
    obs, reward, terminated, truncated, _ = env.step(1)
    done = terminated or truncated
print(obs, reward)
```

8. Code Explanation

The environment owns state and time. It validates actions, applies deterministic dynamics, distinguishes success from a time-limit cutoff, and gives a small step penalty. The loop ends for either termination or truncation, but a learning target can treat them differently.

9. Training / Evaluation

Validate an environment before training: observation shapes and dtypes must match declared spaces, legal actions must be enforced, seeded rollouts must reproduce, rewards must remain within expected scale, and terminal transitions must be correct. Evaluate on held-out scenarios, parameter ranges, or seeds rather than identical training maps.

10. Complexity and Cost

Environment cost ranges from $O(1)$ table lookup to expensive physics simulation. Parallel vectorized environments improve throughput but increase memory. Real-world environments add hardware, latency, safety, and reset costs. Simulator fidelity may matter more than agent compute.

11. Common Use Cases

Gymnasium benchmarks, robot simulators, game engines, digital twins, recommender/user simulators, operations-research simulators, and preference-labeling pipelines.

12. Common Mistakes

- Leaking hidden state into observations
- Mixing termination with time-limit truncation
- Returning inconsistent shapes or units
- Rewarding an easily exploited proxy
- Changing dynamics between runs without versioning
- Evaluating only on training seeds

13. Edge Cases / Limitations

A simulator may have a reality gap. Users and markets are non-stationary. Reset may be impossible in physical systems. Rare safety failures may never appear in training. An apparently Markov observation may omit latent variables, invalidating many assumptions.

14. Variations

Deterministic environments remove transition randomness; stochastic environments model uncertainty. Fully observable environments return state; partially observable ones return observations. Multi-agent environments accept joint actions. Vectorized environments run many copies. Learned environments approximate dynamics for planning.

15. Related Topics

An environment plus agent interface forms an MDP or POMDP. Reward shaping changes the environment's signal. Model-based RL explicitly learns or uses environment dynamics. Sim-to-real methods address mismatch between training simulation and deployment.

16. Interview Questions

1. **What does an RL environment return after a step?** Next observation, reward, termination, truncation, and optional info.
2. **Termination versus truncation?** Termination is intrinsic to the task; truncation is an external cutoff such as a time limit.
3. **What is deterministic dynamics?** Each state-action pair produces one fixed next state.
4. **Why seed an environment?** To reproduce stochastic trajectories and compare algorithms fairly.
5. **What is a reality gap?** Difference between simulated and physical dynamics.
6. **Can reward be part of the observation?** It is feedback returned to the agent, but not normally part of the next state vector.
7. **What makes an observation non-Markov?** It omits information needed to predict future outcomes.
8. **Why validate spaces?** Silent shape/range errors can invalidate training.
9. **What is domain randomization?** Vary simulator parameters to improve robustness.
10. **Known environment versus known model?** You can interact with an environment without having its transition probabilities.

17. Practice Tasks

- Implement and unit-check LineWorld.
- Add stochastic "slip" probability.
- Compare terminal and time-limit target masks.
- Find a reward-hacking strategy in a toy simulator.
- Vectorize 32 independent environment copies.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Grid benchmark suite	Generates configurable navigation tasks	Python, Gymnasium	Procedural mazes	Shows API and testing discipline
Robot digital twin	Simulates balancing/control	PyBullet, PyTorch	Synthetic rollouts	Strong control/sim-to-real signal
User response	Models clicks and delayed satisfaction	Python, Pandas	MovieLens-derived simulator	Demonstrates recommender RL

Project	What it does	Tech stack	Dataset/environment	Resume value
simulator				caveats

19. Quick Revision

- **Key idea:** environment defines dynamics, feedback, observations, and boundaries.
- **Main formula:** $P(s', r \mid s, a)$.
- **When to use:** every RL experiment needs an interaction source.
- **Metrics:** throughput, fidelity, reproducibility, scenario coverage.
- **Common trap:** treating truncation as true terminal state.
- **Interview one-liner:** "The environment advances the world; the agent only chooses actions."

20. Final Cheat Sheet

Item	Summary
Definition	External process responding to actions
Input/output	Action $\rightarrow$ observation, reward, done signals
Main steps	Validate, transition, reward, terminate
Hyperparameters	Horizon, randomness, reward scale
Metrics	Steps/s, fidelity, reproducibility
Pros/cons	Safe scalable simulation / possible reality gap
Best uses	Training and testing sequential decision systems

State

1. Overview

A state is a representation containing enough information to predict the distribution of future outcomes given future actions. In a fully observable Markov decision process, S_t is the true environment state. Agents often receive only an observation O_t , from which they must infer a useful internal state.

2. Intuition

A single photograph of a moving ball does not reveal its velocity. Two recent frames may. A useful state is like a complete game summary: it should preserve information needed for decisions without requiring the entire history.

3. Prerequisites

Conditional probability, Markov property, feature representations, observations, MDPs/POMDPs, and recurrent neural networks for learned memory.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Markov state	Future independent of past given present	Enables Bellman recursion	Board position in chess	State sufficiency
Observation	Signal exposed to agent	May be incomplete/noisy	Camera image	State versus observation
Belief state	Distribution over latent states	Converts POMDP to belief MDP	Robot location probabilities	Bayesian filtering
Feature state	Compressed representation	Controls learnability	CNN embedding of pixels	Representation learning
Terminal state	No task continuation	Stops bootstrap	Goal or failure	Done masks

5. Algorithm / Working Process

1. Environment has latent state S_t .
2. It emits observation $O_t \sim O(\cdot | S_t)$.
3. Agent builds internal state $X_t = f(O_{\leq t}, A_{< t})$, possibly using frame stacking, a filter, or an RNN.
4. Policy and value functions consume X_t .
5. After the action and next observation, memory is updated.

Good state design removes irrelevant detail while retaining decision-relevant information.

6. Mathematical Foundation

The Markov property is

$$\Pr(S_{t+1} | S_0, A_0, \dots, S_t, A_t) = \Pr(S_{t+1} | S_t, A_t).$$

In a POMDP, a belief state is

$$b_t(s) = \Pr(S_t = s | o_{0:t}, a_{0:t-1}),$$

updated by Bayes' rule after actions and observations. State aggregation introduces approximation error when distinct underlying states requiring different actions map to the same representation.

7. Practical Implementation

```
from collections import deque
import numpy as np

class FrameStack:
    def __init__(self, n_frames=4):
        self.frames = deque(maxlen=n_frames)

    def reset(self, observation):
        self.frames.clear()
        for _ in range(self.frames.maxlen):
```

```

        self.frames.append(np.asarray(observation))
    return self.state()

def update(self, observation):
    self.frames.append(np.asarray(observation))
    return self.state()

def state(self):
    return np.stack(self.frames, axis=0)

stack = FrameStack(4)
state = stack.reset(np.array([2.0]))      # position only
state = stack.update(np.array([2.5]))    # history reveals motion
print(state.shape, state.ravel())

```

8. Code Explanation

The wrapper retains the latest four observations. On reset it duplicates the initial observation so shape is stable. Stacking positions lets a network infer direction and speed. This is a simple finite-memory approximation, not a guarantee of Markov sufficiency.

9. Training / Evaluation

Normalize continuous state features using training statistics, preserve categorical semantics, and avoid fitting normalization on evaluation trajectories. Test representation sufficiency by comparing performance with longer histories or recurrent policies. For image states, monitor encoder collapse and augmentation effects.

10. Complexity and Cost

Tabular state storage scales as $O(|S|)$ or $O(|S||A|)$, causing the curse of dimensionality. Frame stacking multiplies observation memory and first-layer compute by the stack length. RNN state is compact but introduces sequential computation and truncated-backpropagation choices.

11. Common Use Cases

Robot pose and velocity, game-board configurations, inventory levels, patient histories, dialogue context, network load, and learned visual embeddings.

12. Common Mistakes

- Treating observation and state as synonyms
- Omitting velocity, time, or history needed for prediction
- Including future information or simulator-only variables
- Using inconsistent normalization at deployment
- Failing to reset recurrent hidden state between episodes
- Building enormous states with irrelevant features

13. Edge Cases / Limitations

No finite observation window may resolve long-term ambiguity. State aliases can map visually identical observations to situations needing different actions. Continuous high-dimensional states make tables impossible. Learned representations can discard rare but safety-critical information.

14. Variations

Tabular states use discrete IDs; factored states use structured variables; pixel states require encoders; belief states model uncertainty; recurrent states summarize history; goal-conditioned states concatenate desired goals. Belief and recurrent approaches are advanced but important in research and real deployments.

15. Related Topics

The **environment** owns latent state, while the agent receives an **observation**. MDPs assume Markov state; POMDPs explicitly model partial observability. Representation learning, RNNs, Transformers, and world models can construct internal states.

16. Interview Questions

1. **Define a Markov state.** It makes the future conditionally independent of earlier history given the current state and action.
2. **State versus observation?** State is sufficient for prediction; observation is merely what the agent senses.
3. **Why stack Atari frames?** One frame lacks motion information such as velocity.
4. **What is a belief state?** A posterior distribution over hidden environment states.
5. **What is state aliasing?** Different latent states share the same representation.
6. **Why is large state space difficult?** Sample and storage requirements grow rapidly.
7. **Can time be part of state?** Yes, especially for finite-horizon tasks where optimal action depends on remaining time.
8. **How do RNNs help?** Their hidden state summarizes observation-action history.
9. **What is a sufficient statistic?** A compression retaining all information needed for relevant future prediction.
10. **When should hidden state reset?** At true episode boundaries, with care for vectorized environments.

17. Practice Tasks

- Construct a Markov state for a moving-object simulator.
- Show failure when velocity is omitted.
- Compare one frame, four frames, and an RNN.
- Detect leakage from a privileged simulator feature.
- Build a belief update for a noisy two-state system.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Partial-observation maze	Compares memory strategies	PyTorch, MiniGrid	Procedural MiniGrid	Strong POMDP experiment

Project	What it does	Tech stack	Dataset/environment	Resume value
Visual control encoder	Learns compact pixel state	torchvision, Gymnasium	CarRacing frames	Representation-learning evidence
Inventory state designer	Tests feature sufficiency	Pandas, PyTorch	Synthetic demand	Business RL framing

19. Quick Revision

- **Key idea:** a state contains decision-relevant predictive information.
- **Main formula:** $P(S_{t+1} | H_t, A_t) = P(S_{t+1} | S_t, A_t)$.
- **When to use:** state is the input to policies, values, and models.
- **Metrics:** return, prediction error, representation size.
- **Common trap:** assuming raw observation is Markov.
- **Interview one-liner:** "State is about sufficient information, not merely current sensor readings."

20. Final Cheat Sheet

Item	Summary
Definition	Markov-sufficient representation of the situation
Input/output	History/observation → policy-ready representation
Main steps	Observe, encode/update memory, act
Hyperparameters	History length, embedding size
Metrics	Downstream return, predictive sufficiency
Pros/cons	Compact decision context / may alias hidden situations
Best uses	Every value, policy, and dynamics model

Action

1. Overview

An action is a decision or control signal issued by the agent that can influence the environment's next state and reward. Action-space structure—discrete, continuous, multidimensional, constrained, or hierarchical—strongly determines which RL algorithms are practical.

2. Intuition

For a self-driving agent, "turn left" is too coarse; steering angle, throttle, and brake are continuous controls. For chess, actions are legal moves from a finite set. The agent needs both a scoring rule and a mechanism that respects what actions are valid.

3. Prerequisites

Probability distributions, categorical and Gaussian sampling, optimization, policy definitions, environment interfaces, and constraints.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Discrete action	One choice from finite set	Supports Q-value enumeration	Up/down/left/right	DQN suitability
Continuous action	Real-valued control	Requires density or deterministic actor	Torque vector	PPO/SAC/DDPG
Multi-discrete	Several categorical choices	Joint space can explode	Select server and mode	Factored policies
Action mask	Invalid choices removed	Prevents illegal behavior	Legal chess moves	Correct masking
Macro-action	Action lasting many steps	Improves long-horizon planning	"Navigate to room"	Hierarchical RL

5. Algorithm / Working Process

The policy produces logits, distribution parameters, Q-values, or a deterministic vector. The agent masks illegal actions, samples or takes the best action, transforms it to environment units, and executes it. Training records the exact selected action and often its log-probability. Continuous actions may be squashed with tanh and rescaled to bounds.

6. Mathematical Foundation

For discrete actions:

$$\pi(a \mid s) = \frac{\exp(z_a)}{\sum_b \exp(z_b)}.$$

For continuous actions, a Gaussian policy may use

$$A \sim \mathcal{N}(\mu_\theta(s), \text{diag}(\sigma_\theta^2(s))).$$

Entropy $H(\pi(\cdot \mid s)) = -\sum_a \pi(a \mid s) \log \pi(a \mid s)$ measures randomness. A bounded action can be $u = L + (U - L)(\tanh x + 1)/2$. Correct policy-gradient likelihoods must include the squashing transformation's Jacobian when required.

7. Practical Implementation

```
import torch

def masked_action(logits, valid_mask, deterministic=False):
```

```

"""Sample a legal discrete action from policy logits."""
if not valid_mask.any():
    raise ValueError("at least one action must be valid")
masked_logits = logits.masked_fill(~valid_mask, float("-inf"))
distribution = torch.distributions.Categorical(logits=masked_logits)
action = masked_logits.argmax() if deterministic else distribution.sample()
return action, distribution.log_prob(action)

logits = torch.tensor([1.2, 3.0, 0.4])
valid = torch.tensor([True, False, True])
action, log_prob = masked_action(logits, valid)
print(action.item(), log_prob.item())

```

8. Code Explanation

Invalid action logits become negative infinity before constructing the categorical distribution, giving them zero probability. Masking after sampling would be biased or could execute illegal actions. The returned log-probability is needed by policy-gradient algorithms.

9. Training / Evaluation

Check action frequencies, entropy, saturation at bounds, illegal-action rate, and per-dimension statistics. During evaluation, deterministic actions improve reproducibility but may hide the behavior of a policy designed to be stochastic. For continuous control, normalize internal actions and consistently map them to physical units.

10. Complexity and Cost

Enumerating discrete choices costs $O(|A|)$, making very large action sets expensive. A factored multidimensional policy can reduce output size from a product to a sum of category sizes, but may miss dependencies. Continuous actors output $O(d)$ parameters for d dimensions, while searching or planning over actions can be much more expensive.

11. Common Use Cases

Game moves, robot torques, routing choices, item recommendations, dialogue responses, treatment choices, bidding, and resource allocations.

12. Common Mistakes

- Using DQN directly for uncountably continuous actions
- Clipping Gaussian samples without understanding likelihood mismatch
- Applying masks after softmax or sampling incorrectly
- Forgetting environment-unit rescaling
- Recording a different action than the one executed
- Ignoring delayed effects of repeated/frame-skipped actions

13. Edge Cases / Limitations

Huge combinatorial spaces make enumeration infeasible. Constraints may depend on state and history. Safety systems can override actions, creating off-policy data. Discontinuous actions make direct pathwise gradients difficult. A poorly chosen action granularity can make the task either impossible or unnecessarily slow.

14. Variations

Discrete policies use categorical distributions; continuous policies use Gaussian, beta, or deterministic actors; hybrid policies combine both. Parameterized actions choose a type plus continuous parameters. Hierarchical policies choose options or goals. Placement interviews emphasize discrete versus continuous algorithm choice.

15. Related Topics

The **policy** defines action probabilities; **Q-values** compare discrete actions; **exploration** perturbs action selection; **actor-critic** learns a parameterized actor; **multi-agent RL** uses joint actions; constrained RL and safe RL restrict feasible behavior.

16. Interview Questions

1. **What is an action space?** The set of decisions the agent may issue.
2. **Why is DQN awkward for continuous actions?** It requires maximizing Q over actions, which cannot be enumerated.
3. **How does a stochastic continuous policy act?** It samples from a density such as a Gaussian parameterized by the network.
4. **What is action masking?** Setting invalid actions to zero probability before selection.
5. **Why store log-probability?** Policy-gradient ratios and losses need the probability of the executed action.
6. **Clipping versus squashing?** Clipping creates boundary mass and likelihood mismatch; tanh gives a differentiable bounded transform.
7. **What is a deterministic policy?** A direct mapping $a = \mu(s)$.
8. **What is action repeat?** Executing one selected action for several environment frames.
9. **Why normalize continuous controls?** It improves optimization and gives consistent network output scales.
10. **What is a macro-action?** A temporally extended decision composed of lower-level actions.

17. Practice Tasks

- Implement epsilon-greedy selection with random tie-breaking.
- Add legal-action masks to a grid game.
- Rescale tanh outputs to arbitrary control bounds.
- Diagnose a policy stuck at action limits.
- Compare joint versus factored multidiscrete policies.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Masked board-game agent	Selects only legal moves	PyTorch, PettingZoo	Connect Four	Shows correct policy constraints
Continuous car controller	Produces steering/throttle	PyTorch, Gymnasium	CarRacing	Demonstrates continuous control
Hierarchical navigator	Chooses rooms then motion	PyTorch, MiniGrid	Procedural maps	Advanced long-horizon project

19. Quick Revision

- **Key idea:** actions are the agent's controls over future outcomes.
- **Main formula:** $\pi(a | s) = \text{softmax}(z)_a$ for discrete control.
- **When to use:** every interaction step.
- **Metrics:** return, entropy, invalid-action rate, bound saturation.
- **Common trap:** selecting an algorithm incompatible with the action space.
- **Interview one-liner:** "Action-space geometry determines how a policy is represented and optimized."

20. Final Cheat Sheet

Item	Summary
Definition	Agent-issued decision/control
Input/output	Policy state $\rightarrow$ discrete choice or continuous vector
Main steps	Score, mask, sample/select, rescale, execute
Hyperparameters	Entropy, noise scale, action repeat
Metrics	Return, legality, entropy, saturation
Pros/cons	Direct control / combinatorial or constrained spaces are hard
Best uses	Decision-making and control tasks

Reward

1. Overview

A reward is a scalar feedback signal emitted after an action. It describes what the designer wants, while the RL algorithm determines how to achieve it. The agent maximizes cumulative return, so reward timing, scale, loopholes, and discounting matter as much as the immediate number.

2. Intuition

Reward is like a game score, not a step-by-step instruction. Giving +1 only at a maze exit states the goal but makes learning difficult. Giving +1 for moving toward the exit provides guidance but may create a loophole if the agent can repeatedly move away and back.

3. Prerequisites

Random variables, expectation, discounted sums, MDPs, objectives, terminal states, and basic optimization.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Immediate reward	One-step scalar R_{t+1}	Local feedback	+1 for scoring	Reward versus return
Return	Accumulated future reward	Actual optimization target	Total game score	Discounting
Sparse/dense	Rare versus frequent signal	Changes learning difficulty	Goal-only versus distance	Reward shaping
Extrinsic/intrinsic	Task signal versus exploration bonus	Helps discovery	Novel-state bonus	Exploration
Reward hacking	Exploiting a misspecified proxy	Major safety concern	Looping for points	Alignment

5. Algorithm / Working Process

The environment computes reward from the transition. The agent combines current and future rewards into a return or bootstrapped target, estimates how behavior affects that return, and updates its policy. Reward transformations such as clipping or normalization may improve stability but can change the task. Evaluation should use the original task reward.

6. Mathematical Foundation

The discounted return is

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = R_{t+1} + \gamma G_{t+1}.$$

For episodic tasks, $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$. Discounting ensures finite returns in continuing tasks and prioritizes earlier reward. Potential-based shaping,

$$F(s, a, s') = \gamma \Phi(s') - \Phi(s),$$

preserves the set of optimal policies under standard conditions, unlike arbitrary shaping.

7. Practical Implementation

```
def discounted_returns(rewards, gamma):
    returns, running = [], 0.0
    for reward in reversed(rewards):
        running = reward + gamma * running
```

```
returns.append(running)
return list(reversed(returns))

rewards = [-0.1, -0.1, 1.0]
print(discounted_returns(rewards, gamma=0.9))
# [0.62, 0.8, 1.0]
```

8. Code Explanation

Backward recursion implements $G_t = R_{t+1} + \gamma G_{t+1}$ in $O(T)$. The last reward has no later continuation. Each earlier position receives discounted credit for the terminal success.

9. Training / Evaluation

Inspect reward distributions, individual reward components, episode length, and correlations between reward and intended success. Normalize advantages rather than silently redefining the reported reward. Use held-out scenarios and human review for proxy gaming. In multi-objective tasks, report each component instead of only their weighted sum.

10. Complexity and Cost

Reward calculation is usually $O(1)$, while computing full returns is $O(T)$ per trajectory. Sparse reward can multiply interaction cost dramatically. Learned reward models introduce labeling, training, inference, uncertainty, and adversarial-optimization costs.

11. Common Use Cases

Game score, task completion, energy efficiency, user satisfaction, portfolio utility, robot progress, safety penalties, and human preference scores.

12. Common Mistakes

- Confusing reward with return
- Choosing reward scale without retuning learning rate/value loss
- Rewarding a proxy that is easy to exploit
- Adding dense shaping that changes the optimal policy
- Clipping reward during evaluation
- Treating absence of reward as evidence an action was bad

13. Edge Cases / Limitations

Delayed and sparse rewards make credit assignment hard. Conflicting objectives require trade-offs. Human preferences may be noisy and biased. Some goals cannot be safely reduced to one scalar. A fixed reward may become invalid as users or environments change.

14. Variations

Binary, continuous, vector-valued, constrained, risk-sensitive, intrinsic, learned, and preference-based rewards are common. Reward redistribution moves delayed feedback earlier. Potential-based shaping is especially

interview-relevant because it offers a policy-invariance guarantee.

15. Related Topics

Return aggregates rewards; **value functions** predict return; **reward models** power RLHF; **exploration bonuses** create intrinsic reward; **inverse RL** infers reward from demonstrations.

16. Interview Questions

1. **Reward versus return?** Reward is one-step feedback; return is accumulated future reward.
2. **Why discount?** To control horizon, prefer earlier outcomes, and make continuing sums finite.
3. **What does $\gamma = 0$ imply?** The agent is myopic and values only the next reward.
4. **What is sparse reward?** Informative feedback occurs rarely.
5. **What is reward shaping?** Adding intermediate signals to accelerate learning.
6. **Can shaping change optimal behavior?** Yes, unless designed with guarantees such as potential-based shaping.
7. **What is reward hacking?** Maximizing the metric in an unintended way.
8. **Why normalize reward?** It can improve numerical conditioning, but must preserve evaluation semantics.
9. **What is intrinsic reward?** A bonus generated from novelty, prediction error, or curiosity.
10. **Why is reward design difficult?** The scalar must encode long-term, multi-objective, and safety preferences without loopholes.

17. Practice Tasks

- Compute discounted returns by hand and in Python.
- Compare $\gamma = 0.5, 0.9, 0.99$ on delayed reward.
- Design and exploit a flawed shaping reward.
- Implement potential-based shaping for a gridworld.
- Plot separate reward components during training.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Reward-hacking benchmark	Demonstrates proxy loopholes	Python, Gymnasium	Custom grid tasks	Shows AI safety insight
Preference reward model	Predicts pairwise choices	PyTorch	Synthetic/human rankings	Connects RLHF foundations
Sparse-reward navigator	Compares shaping methods	PyTorch, MiniGrid	MiniGrid	Strong experimental design

19. Quick Revision

- **Key idea:** reward specifies local feedback; return is optimized.
- **Main formula:** $G_t = R_{t+1} + \gamma G_{t+1}$.

- **When to use:** define objectives and training signals.
- **Metrics:** task return, success, component scores, violations.
- **Common trap:** creating an exploitable proxy.
- **Interview one-liner:** "Reward says what is desirable; return turns it into a long-horizon objective."

20. Final Cheat Sheet

Item	Summary
Definition	Scalar feedback from a transition
Input/output	Transition $\rightarrow$ scalar
Main steps	Compute, accumulate/bootstrapped target, optimize
Hyperparameters	Scale, discount, component weights
Metrics	Episodic return, success, safety
Pros/cons	Simple objective signal / hard to specify safely
Best uses	Sequential goals with measurable feedback

Policy

1. Overview

A policy defines how an agent chooses actions. A stochastic policy maps each state to a probability distribution $\pi(a \mid s)$; a deterministic policy maps it to one action $\mu(s)$. The policy may be a table, rule, linear model, neural network, or planning procedure.

2. Intuition

A navigation policy is a driving habit: given the current situation, it says how likely the driver is to turn, accelerate, or brake. Training improves the habit; exploration keeps it from always making the currently favored choice.

3. Prerequisites

Conditional probability, softmax and Gaussian distributions, neural networks, optimization, expected return, and value functions.

4. Core Concepts

Subtopic	What it means	Why it matters	Example	Common interview angle
Stochastic policy	Distribution over actions	Exploration and multimodality	Softmax move choice	Policy entropy

Subtopic	What it means	Why it matters	Example	Common interview angle
Deterministic policy	One action per state	Efficient continuous control	Steering mean	DDPG-style methods
Behavior policy	Collects data	Determines data distribution	Epsilon-greedy policy	Off-policy learning
Target policy	Being evaluated/improved	May differ from behavior	Greedy policy	Importance sampling
Parameterization	Table/model with θ	Enables generalization	Neural actor	Logits versus probabilities

5. Algorithm / Working Process

Given state features, a policy network computes action scores or distribution parameters. Invalid actions are masked. Training samples an action and saves its log-probability; evaluation commonly selects the mode or mean. A policy-gradient update raises probability of actions with positive advantage and lowers probability of actions with negative advantage.

6. Mathematical Foundation

For finite actions,

$$\pi_{\theta}(a \mid s) = \text{softmax}(f_{\theta}(s))_a.$$

The policy objective is $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[G_0]$. The policy-gradient theorem gives

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) Q^{\pi_{\theta}}(S_t, A_t)].$$

Subtracting a state-dependent baseline does not bias the expected gradient and reduces variance.

7. Practical Implementation

```
import torch
from torch import nn

class DiscretePolicy(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 64), nn.Tanh(),
            nn.Linear(64, n_actions)
        )

    def distribution(self, states):
        return torch.distributions.Categorical(logits=self.net(states))

    def act(self, state, deterministic=False):
        dist = self.distribution(state)
```

```

        action = dist.probs.argmax(-1) if deterministic else dist.sample()
        return action, dist.log_prob(action)

policy = DiscretePolicy(4, 2)
action, log_prob = policy.act(torch.zeros(1, 4))
print(action.item(), log_prob.item())

```

8. Code Explanation

The network outputs logits, not already-normalized probabilities. “Categorical” performs stable normalization and supports sampling, log-probability, and entropy. The same policy exposes stochastic training and deterministic evaluation behavior.

9. Training / Evaluation

Track episodic return, entropy, approximate KL divergence after updates, action frequencies, and gradient norms. Evaluate fixed checkpoints over multiple seeds without learning. For stochastic tasks, report both deterministic and sampled-policy results when deployment will sample.

10. Complexity and Cost

A tabular policy costs $O(|S||A|)$ memory. A neural policy costs one forward pass per action and one backward pass per training batch. Discrete output cost grows with $|A|$; Gaussian continuous policies usually output $2d$ parameters for d action dimensions.

11. Common Use Cases

Robot controllers, game-playing strategies, recommendation and ranking policies, resource allocation, dialogue management, and LLM token-generation policies.

12. Common Mistakes

- Feeding probabilities into an API expecting logits
- Taking log of zero or computing softmax unstably
- Forgetting action masks
- Using deterministic evaluation during supposedly stochastic deployment
- Updating too far from the data-collecting policy
- Confusing behavior and target policies

13. Edge Cases / Limitations

Policies can collapse prematurely to one action. Gaussian policies may poorly represent bounded or multimodal control. Huge action vocabularies make full softmax expensive. A reactive policy cannot solve tasks requiring memory unless state includes it.

14. Variations

Tabular, epsilon-greedy, Boltzmann, categorical, Gaussian, beta, deterministic, recurrent, goal-conditioned, hierarchical, and autoregressive policies are common. Autoregressive policies are critical for LLM RLHF;

recurrent policies matter for POMDPs.

15. Related Topics

Policy gradients optimize the policy directly. **Actor-critic** uses a critic to estimate advantages. **Q-learning** represents a greedy policy implicitly through Q-values. **PPO** constrains policy updates.

16. Interview Questions

1. **What is a policy?** A mapping from state information to action choices or probabilities.
2. **Stochastic versus deterministic?** One samples a distribution; the other returns a single action.
3. **What are logits?** Unnormalized scores whose softmax produces categorical probabilities.
4. **Why use stochastic policies?** Exploration, uncertain environments, and multimodal optimal behavior.
5. **Behavior versus target policy?** One generates data; the other is learned or evaluated.
6. **What is policy entropy?** Expected uncertainty of action choices.
7. **Why compute log-probability?** Its gradient yields the likelihood-ratio policy estimator.
8. **Can the optimal policy be deterministic?** In a finite fully observed MDP, at least one deterministic optimal policy exists; constraints or partial observability can favor stochasticity.
9. **What is policy collapse?** Entropy falls too fast and exploration disappears.
10. **Why constrain policy updates?** Large shifts make on-policy estimates unreliable and can destroy performance.

17. Practice Tasks

- Implement a softmax policy from logits.
- Add entropy regularization.
- Compare sampled and greedy evaluation.
- Debug a double-softmax implementation.
- Build a recurrent policy for partial observations.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Policy visualizer	Shows action probabilities over a grid	PyTorch, Streamlit	GridWorld	Explainable RL artifact
Recurrent memory policy	Solves hidden-cue tasks	PyTorch, Gymnasium	Memory environment	Demonstrates POMDP skill
Token policy sandbox	Optimizes short sequences	PyTorch	Synthetic sequence rewards	Bridges RL and LLMs

19. Quick Revision

- **Key idea:** policy is the agent's action-selection rule.
- **Main formula:** $\pi_{\theta}(a | s) = \text{softmax}(f_{\theta}(s))_a$.
- **When to use:** choose actions at every step.

- **Metrics:** return, entropy, KL, action frequencies.
- **Common trap:** confusing logits, probabilities, and log-probabilities.
- **Interview one-liner:** "A policy converts state into behavior; RL improves that behavior for long-term return."

20. Final Cheat Sheet

Item	Summary
Definition	State-conditioned action mapping/distribution
Input/output	State $\rightarrow$ action or action distribution
Main steps	Encode, score, mask, sample/select
Hyperparameters	Temperature, entropy weight, network size
Metrics	Return, entropy, KL
Pros/cons	Direct behavior representation / can collapse or shift unstably
Best uses	All action-selection problems

Value Function

1. Overview

A value function predicts expected future return. The state-value $V^\pi(s)$ measures how good it is to be in state s while following policy π . It supports planning, policy evaluation, bootstrapping, variance reduction, and actor-critic learning.

2. Intuition

In chess, the immediate reward is usually zero until the game ends, but a value function can say a position is promising. It compresses all possible future continuations under a policy into one number.

3. Prerequisites

Expected value, conditional probability, returns, policies, MDPs, Bellman equations, regression, and temporal-difference learning.

4. Core Concepts

Subtopic	What it means	Why it matters	Example	Common interview angle
State-value	Expected return from state	Evaluates situations	Winning probability proxy	V^π definition
Policy dependence	Value assumes future policy	Different policies give different values	Aggressive versus safe play	V^π versus V^*

Subtopic	What it means	Why it matters	Example	Common interview angle
Bootstrapping	Learn from another estimate	Data-efficient updates	$r + \gamma V(s')$	Bias-variance
Monte Carlo target	Complete sampled return	Unbiased but high variance	Episode score	MC versus TD
Function approximation	Generalize across states	Handles large spaces	Neural critic	Deadly triad

5. Algorithm / Working Process

Collect transitions under a policy. Build targets using complete returns, one-step TD, or multi-step returns. Predict $V_\phi(s)$, compute the target error, and update ϕ by regression. In actor-critic, use the critic's TD error or advantage estimate to update the policy.

6. Mathematical Foundation

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

The Bellman expectation equation is

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma V^\pi(s')].$$

One-step TD error:

$$\delta_t = R_{t+1} + \gamma(1 - d_t)V_\phi(S_{t+1}) - V_\phi(S_t).$$

A common critic loss is $\mathcal{L}_V = \frac{1}{2} \mathbb{E}[(V_\phi(S_t) - y_t)^2]$, with the target detached from gradient computation.

7. Practical Implementation

```
import torch
from torch import nn

critic = nn.Sequential(nn.Linear(4, 64), nn.ReLU(), nn.Linear(64, 1))
optimizer = torch.optim.Adam(critic.parameters(), lr=3e-4)

states = torch.randn(32, 4)
next_states = torch.randn(32, 4)
rewards = torch.randn(32)
terminated = torch.zeros(32)
gamma = 0.99

values = critic(states).squeeze(-1)
with torch.no_grad():
    targets = rewards + gamma * (1 - terminated) * critic(next_states).squeeze(-1)
loss = 0.5 * (values - targets).pow(2).mean()
optimizer.zero_grad()
```

```
loss.backward()  
optimizer.step()  
print(float(loss))
```

8. Code Explanation

The critic maps each state to one scalar. The no-gradient block makes the bootstrapped target fixed for this optimization step. The termination mask prevents adding nonexistent continuation value after a terminal transition.

9. Training / Evaluation

Use return prediction error, explained variance, calibration, and downstream policy return. Random transition splits can leak adjacent trajectory information; evaluate on held-out episodes or scenarios. Watch for target drift, exploding value scale, and overfitting to a replay distribution.

10. Complexity and Cost

Tabular value storage is $O(|S|)$. Exact dynamic-programming sweeps cost roughly $O(|S|^2|A|)$ for dense transitions. Neural value inference is one forward pass per state; training costs a forward/backward pass and target computation.

11. Common Use Cases

Critics in PPO/A2C, search leaf evaluation, game position scoring, baseline subtraction in policy gradients, bootstrapped control, and terminal-outcome prediction.

12. Common Mistakes

- Omitting policy dependence from V^π
- Bootstrapping through true terminal states
- Stopping bootstrap on time-limit truncations without thought
- Backpropagating through the target network unintentionally
- Comparing raw value loss across different reward scales
- Treating low critic loss as proof of a good policy

13. Edge Cases / Limitations

A scalar expectation hides risk and multimodal returns. Off-policy bootstrapping with nonlinear function approximation can diverge. Values are unreliable on out-of-distribution states. Long horizons and sparse rewards propagate information slowly.

14. Variations

Optimal value V^* , finite-horizon V_t , distributional value, quantile value, risk-sensitive value, goal-conditioned value, and successor features extend the basic expectation. Distributional value is research-relevant; ordinary V^π is placement-essential.

15. Related Topics

Q-values condition on the first action. **Bellman equations** recursively characterize values. **Advantages** compare an action with state baseline. **Actor-critic** trains a value critic alongside a policy.

16. Interview Questions

1. **Define $V^\pi(s)$.** Expected discounted return from s when following π .
2. **Why is value policy-dependent?** Future action choices determine future rewards.
3. **What is V^* ?** Maximum achievable expected return from each state.
4. **What is bootstrapping?** Updating an estimate using another current estimate.
5. **MC versus TD?** MC uses complete returns; TD uses bootstrapped partial returns.
6. **Why mask terminals?** Terminal states have no continuation return.
7. **What is critic explained variance?** How much variance in targets is captured by value predictions.
8. **Why detach TD targets?** To make semi-gradient learning update the current prediction only.
9. **Can value loss decrease while policy worsens?** Yes; the critic may fit data from a poor or shifting policy.
10. **Why does a critic reduce variance?** It provides a state-dependent baseline for policy updates.

17. Practice Tasks

- Fit tabular V^π by iterative policy evaluation.
- Compare Monte Carlo and TD(0) learning curves.
- Test terminal versus truncation masks.
- Plot predicted value along successful trajectories.
- Implement n-step value targets.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Chess-position critic	Predicts future outcome	PyTorch	Lichess positions/outcomes	Strong supervised-to-RL bridge
TD versus MC lab	Compares bias and variance	NumPy, Gymnasium	Random Walk	Interview-ready analysis
Distributional critic	Predicts return quantiles	PyTorch	CartPole variants	Research-oriented extension

19. Quick Revision

- **Key idea:** value predicts future return from a state under a policy.
- **Main formula:** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$.
- **When to use:** evaluation, bootstrapping, planning, policy baselines.
- **Metrics:** MSE, explained variance, calibration, policy return.
- **Common trap:** mishandling terminal bootstrap.
- **Interview one-liner:** "A value function turns delayed reward into a learnable prediction at every state."

20. Final Cheat Sheet

Item	Summary
Definition	Expected return conditioned on state
Input/output	State $\rightarrow$ scalar
Main steps	Build target, predict, regress
Hyperparameters	γ , step size, n-step horizon
Metrics	Value loss, explained variance
Pros/cons	Dense learning signal / biased or OOD estimates
Best uses	Critics, planning, policy evaluation

Q-Value

1. Overview

The action-value or Q-value $Q^\pi(s, a)$ is the expected return after taking action a in state s and then following policy π . It directly compares choices, making it central to Q-learning, SARSA, DQN, and many offline RL methods.

2. Intuition

A state-value says how good a restaurant visit is overall; Q-values say how good each menu choice is from that situation. Selecting the largest Q-value yields a greedy action.

3. Prerequisites

State values, policies, returns, conditional expectation, Bellman equations, tabular arrays, and neural regression.

4. Core Concepts

Subtopic	What it means	Why it matters	Example	Common interview angle
Q^π	Return after fixed first action, then π	Evaluates choices	Value of "move right"	Relation to V^π
Q^*	Best achievable action-value	Defines optimal control	Best path action	Greedy optimal policy
Greedy selection	$\arg \max_a Q(s, a)$	Converts values to behavior	Highest-valued arm	Ties/exploration
TD target	Reward plus next Q estimate	Learns without full episode	Q-learning target	On/off-policy

Subtopic	What it means	Why it matters	Example	Common interview angle
Overestimation	Max of noisy estimates is biased high	Destabilizes learning	DQN value inflation	Double Q-learning

5. Algorithm / Working Process

Observe s , select and execute a , observe r , s' , build a target from reward plus future action-value, and regress $Q(s, a)$ toward it. At decision time, choose the best action or explore. Tabular methods update one entry; DQN updates neural parameters from batches.

6. Mathematical Foundation

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Relationship to state value:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a).$$

Bellman expectation:

$$Q^\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid s, a].$$

Optimality:

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid s, a].$$

7. Practical Implementation

```
import numpy as np

q = np.zeros((5, 2))
state, action = 1, 1
reward, next_state, terminated = -0.01, 2, False
alpha, gamma = 0.1, 0.99

bootstrap = 0.0 if terminated else np.max(q[next_state])
target = reward + gamma * bootstrap
q[state, action] += alpha * (target - q[state, action])
print(q[state])
```

8. Code Explanation

The table has one row per state and one column per action. The selected entry moves toward a one-step optimality target. "target - current" is the TD error. The max makes this a Q-learning-style target rather than SARSA.

9. Training / Evaluation

Track return, TD error distribution, max-Q scale, policy action coverage, and calibration against sampled returns. A low TD loss can coexist with poor behavior if data lacks useful actions. For offline data, evaluate conservatively because unseen actions can receive arbitrarily high estimates.

10. Complexity and Cost

Tabular memory is $O(|S||A|)$, lookup is $O(1)$, and greedy choice is $O(|A|)$. A DQN forward pass outputs all discrete-action values. Continuous-action maximization is generally nontrivial, motivating actor networks.

11. Common Use Cases

Discrete games, navigation, scheduling, recommendation with manageable candidate sets, off-policy control, and critics for deterministic actors.

12. Common Mistakes

- Forgetting that Q^π includes a specified future policy
- Using max target while claiming the update is SARSA
- Bootstrapping terminal transitions
- Treating Q-values as probabilities
- Using DQN unchanged for continuous actions
- Trusting high Q-values on unsupported offline actions

13. Edge Cases / Limitations

Large or continuous action spaces make argmax costly. Function approximation can generalize incorrectly. The maximum creates overestimation bias. Partial observability makes state-action values history-dependent. Offline extrapolation error is especially severe.

14. Variations

Double Q-values reduce overestimation; dueling networks separately estimate value and advantage; distributional Q predicts a return distribution; quantile critics support risk sensitivity; conservative Q-learning penalizes unsupported actions.

15. Related Topics

Value averages Q under a policy. **Advantage** is $A^\pi = Q^\pi - V^\pi$. **Q-learning** targets a greedy next action, **SARSA** targets the next behavior action, and **DQN** uses a neural Q approximator.

16. Interview Questions

1. **Define** $Q^\pi(s, a)$. Expected return after a in s , then following π .
2. **How is V^π obtained from Q?** Average Q-values using $\pi(a | s)$.
3. **How does Q define a greedy policy?** Choose $\arg \max_a Q(s, a)$.
4. **Are Q-values probabilities?** No; they use reward units and can be any real value.
5. **What causes max overestimation?** Maximizing noisy estimates favors positive errors.
6. **How does Double Q help?** It separates action selection from action evaluation.
7. **Q-learning versus SARSA target?** Max next Q versus Q of the actually selected next action.

8. **Why are Q-values difficult offline?** Unseen actions lack grounded targets but may be selected by max.
9. **Can Q handle continuous actions?** Yes in principle, but maximizing over actions requires optimization or an actor.
10. **What is a dueling Q network?** A network combining state value and centered action advantages.

17. Practice Tasks

- Implement a tabular Q update.
- Verify $V^\pi = \sum_a \pi Q^\pi$.
- Demonstrate max overestimation with noisy estimates.
- Add Double Q-learning.
- Plot Q-values and the greedy policy on a grid.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Q-map debugger	Visualizes values and TD errors	NumPy, Plotly	GridWorld	Excellent interpretability demo
Double-Q experiment	Measures overestimation	PyTorch	CartPole/noisy MDP	Strong algorithm analysis
Conservative recommender	Learns from logged choices	PyTorch	MovieLens bandit logs	Offline RL relevance

19. Quick Revision

- **Key idea:** Q predicts return for a state-action pair.
- **Main formula:** $Q^\pi(s, a) = \mathbb{E}[G_t \mid s, a]$.
- **When to use:** compare actions and derive discrete control policies.
- **Metrics:** return, TD error, Q calibration, overestimation.
- **Common trap:** treating Q as probability or ignoring policy dependence.
- **Interview one-liner:** "Q-values attach long-term consequences directly to candidate actions."

20. Final Cheat Sheet

Item	Summary
Definition	Expected return conditioned on state and first action
Input/output	State-action $\rightarrow$ scalar; DQN state $\rightarrow$ all action values
Main steps	Observe transition, target, TD update, greedy/explore
Hyperparameters	α, γ, ϵ , target-update rate
Metrics	Return, TD loss, Q scale

Item	Summary
Pros/cons	Direct action ranking / max bias and large-action cost
Best uses	Discrete off-policy control

Markov Decision Process

1. Overview

A Markov Decision Process (MDP) is the standard mathematical model for sequential decision-making under uncertainty. It specifies states, actions, transition dynamics, rewards, and discounting. Most RL algorithms assume the task is an MDP even when the dynamics and reward function are unknown.

2. Intuition

A board game is an MDP when the board position contains everything needed to determine possible future positions and outcomes. The history of how the board arose is irrelevant once the current board is known. The agent chooses moves; rules and randomness determine the next position and reward.

3. Prerequisites

Conditional probability, expectation, Markov chains, states/actions/rewards, policies, discounted returns, and basic linear algebra.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
State set $\mathcal{S}$	Possible Markov situations	Domain of value functions	Grid cells	State design
Action set $\mathcal{A}$	Available decisions	Defines control	Four directions	State-dependent actions
Transition P	Next-state probabilities	Encodes dynamics	10% chance of slip	Model known/unknown
Reward R	Transition utility	Defines objective	+1 at goal	Reward design
Discount γ	Future weighting	Sets effective horizon	0.99	Continuing tasks
Initial distribution ρ_0	Episode start states	Affects objective	Random spawn	Train/eval coverage

5. Algorithm / Working Process

1. Sample $S_0 \sim \rho_0$.

2. Agent samples $A_t \sim \pi(\cdot \mid S_t)$.
3. Environment samples $(S_{t+1}, R_{t+1}) \sim p(\cdot, \cdot \mid S_t, A_t)$.
4. Continue until termination or indefinitely.
5. Evaluate the policy by expected return and improve it using values, policy gradients, or planning.

If P and R are known, dynamic programming or planning can solve the MDP. If unknown, RL estimates useful quantities from samples.

6. Mathematical Foundation

An MDP is often written

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0).$$

The Markov property is

$$\Pr(S_{t+1}, R_{t+1} \mid S_{0:t}, A_{0:t}) = \Pr(S_{t+1}, R_{t+1} \mid S_t, A_t).$$

Policy performance is

$$J(\pi) = \mathbb{E}_{S_0 \sim \rho_0, \tau \sim \pi}[G_0].$$

For finite discounted MDPs, Bellman optimality operators are contractions under the sup norm, giving unique optimal values and convergence of exact value iteration.

7. Practical Implementation

```
import numpy as np

# Two states, two actions. P[s, a, s_next], R[s, a].
P = np.array([
    [[1.0, 0.0], [0.2, 0.8]],
    [[0.0, 1.0], [0.0, 1.0]],
])
R = np.array([[0.0, 1.0], [0.0, 0.0]])
gamma = 0.9

V = np.zeros(2)
for _ in range(100):
    q = R + gamma * np.einsum("sak,k->sa", P, V)
    new_V = q.max(axis=1)
    if np.max(np.abs(new_V - V)) < 1e-10:
        break
    V = new_V

policy = q.argmax(axis=1)
print("optimal value:", V, "policy:", policy)
```

8. Code Explanation

The transition tensor stores next-state probabilities. “einsum” computes expected next value for every state-action pair. Repeated Bellman optimality backups converge to V^* ; taking argmax produces a deterministic optimal policy.

9. Training / Evaluation

For a specified MDP, verify every transition row sums to one, rewards and terminals match the problem, and initial-state coverage reflects deployment. Evaluate return from ρ_0 , not arbitrary states only. With learned models, also report dynamics error and policy performance because low one-step model error need not imply good control.

10. Complexity and Cost

Dense value iteration costs $O(K|\mathcal{S}|^2|\mathcal{A}|)$ for K sweeps and stores $O(|\mathcal{S}|^2|\mathcal{A}|)$ transitions. Sparse transitions reduce cost. Continuous/high-dimensional MDPs require approximation, sampling, or planning rather than explicit tensors.

11. Common Use Cases

Inventory control, routing, games, robot control, treatment planning, maintenance, queueing, resource allocation, and formal benchmarks for RL algorithms.

12. Common Mistakes

- Claiming a process is Markov without checking state sufficiency
- Omitting the initial-state distribution from the objective
- Confusing known environment access with known transition model
- Treating truncation as an MDP terminal event
- Assuming rewards depend only on state when they may depend on transition
- Applying finite-state dynamic programming to huge continuous spaces

13. Edge Cases / Limitations

Real observations may be partially observed, producing a POMDP. Environments can be non-stationary. Multi-agent learning changes effective dynamics. Long horizons make discount choice influential. A fully specified exact MDP is often unavailable in real systems.

14. Variations

Finite-horizon MDPs use time-dependent values; average-reward MDPs avoid discounting; constrained MDPs add cost limits; POMDPs include observations and hidden states; semi-MDPs support temporally extended actions; stochastic games generalize to multiple agents.

15. Related Topics

Bellman equations exploit the Markov property. **Model-based RL** learns or uses P, R . **Offline RL** learns from fixed MDP trajectories. **Multi-agent RL** replaces one controlled MDP with a stochastic game.

16. Interview Questions

1. **What is an MDP?** A tuple defining states, actions, transitions, rewards, discount, and usually initial-state distribution.
2. **What is the Markov property?** Current state and action are sufficient for the next outcome distribution.
3. **Known model versus model-free RL?** Known model exposes P, R ; model-free methods learn values/policies from samples.
4. **What is an effective horizon?** Roughly $1/(1 - \gamma)$ for continuing discounted problems.
5. **Does an optimal deterministic policy exist?** For finite discounted MDPs, yes.
6. **MDP versus Markov chain?** An MDP includes controlled actions and rewards; a Markov chain has fixed transitions.
7. **What is ρ_0 ?** Distribution over starting states.
8. **Why is state sufficiency important?** Bellman recursion relies on it.
9. **Finite-horizon difference?** Optimal policy/value may depend explicitly on remaining time.
10. **What is a POMDP?** An MDP whose true state is hidden and only noisy/incomplete observations are available.

17. Practice Tasks

- Specify a gridworld as an MDP tuple.
- Check transition normalization automatically.
- Solve a small MDP with value iteration.
- Add finite horizon and compare policies by time step.
- Convert a hidden-state example into a belief-state MDP.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
MDP solver	Runs evaluation, iteration, visualization	NumPy, Streamlit	Configurable finite MDPs	Strong fundamentals demo
Inventory controller	Minimizes stockout/holding cost	Python, SciPy	Synthetic demand	Business optimization value
Treatment simulator	Chooses sequential interventions	PyTorch/NumPy	Synthetic patient MDP	Shows responsible modeling

19. Quick Revision

- **Key idea:** MDP formalizes controlled Markov dynamics and reward.
- **Main formula:** $\Pr(s', r \mid \text{history}, a) = \Pr(s', r \mid s, a)$.
- **When to use:** formalize sequential decisions.
- **Metrics:** expected return, constraint costs, robustness.
- **Common trap:** using a non-Markov observation as state.
- **Interview one-liner:** "An MDP is the contract that makes recursive value reasoning possible."

20. Final Cheat Sheet

Item	Summary
Definition	$(\mathcal{S}, \mathcal{A}, P, R, \gamma, \rho_0)$
Input/output	State/action $\rightarrow$ next-state/reward distribution
Main steps	Start, act, transition, reward, repeat
Hyperparameters	γ , horizon
Metrics	Expected return, success, costs
Pros/cons	Clean theory / full observability and stationarity assumptions
Best uses	Modeling and solving sequential control

Exploration vs Exploitation

1. Overview

Exploration versus exploitation is the trade-off between trying uncertain actions to gain information and choosing the action currently believed to be best. Pure exploitation can lock into a poor policy; excessive exploration wastes reward and may be unsafe.

2. Intuition

When choosing a restaurant, exploitation means returning to your favorite. Exploration means trying a new place that might be better. Early on, exploration has high information value; later, reliable knowledge usually justifies more exploitation.

3. Prerequisites

Expected reward, uncertainty, bandits, policies, Q-values, probability distributions, regret, and basic confidence intervals.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Epsilon-greedy	Random action with probability ϵ	Simple broad coverage	10% random moves	Decay schedule
Softmax exploration	Sample by value-derived probabilities	Prefers promising alternatives	Temperature policy	Scale sensitivity
Optimism/UCB	Add uncertainty bonus	Directed exploration	Confidence-bound bandit	Regret

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Entropy bonus	Reward stochastic policy	Prevents early collapse	PPO entropy term	Coefficient tuning
Intrinsic motivation	Reward novelty/learning progress	Helps sparse reward	Prediction-error bonus	Noisy-TV problem

5. Algorithm / Working Process

Estimate action quality and optionally uncertainty. Apply an exploration rule to choose actions. Update values from observed outcomes. Reduce exploration only when estimates become reliable, not merely because wall-clock steps passed. During evaluation, remove training-time random exploration unless stochastic behavior is part of the deployed policy.

6. Mathematical Foundation

Epsilon-greedy:

$$A_t = \begin{cases} \text{uniform random action,} & \text{probability } \epsilon, \\ \arg \max_a Q(S_t, a), & \text{probability } 1 - \epsilon. \end{cases}$$

Boltzmann policy:

$$\pi(a \mid s) = \frac{\exp(Q(s, a)/\tau)}{\sum_b \exp(Q(s, b)/\tau)}.$$

UCB1 chooses

$$A_t = \arg \max_a \left[\hat{\mu}_a + c \sqrt{\frac{\log t}{N_a}} \right].$$

Cumulative regret compares obtained reward with an oracle that always chooses the optimal action.

7. Practical Implementation

```
import numpy as np

def ucb_action(value_estimates, counts, step, c=2.0):
    untried = np.flatnonzero(counts == 0)
    if len(untried):
        return int(untried[0])
    bonus = c * np.sqrt(np.log(step) / counts)
    return int(np.argmax(value_estimates + bonus))

values = np.array([0.4, 0.6, 0.5])
counts = np.array([20, 100, 5])
print(ucb_action(values, counts, step=125))
```

8. Code Explanation

Each action is tried once to avoid division by zero. Thereafter, the action score combines estimated mean reward with a bonus that is larger for under-sampled actions. The bonus shrinks as evidence accumulates.

9. Training / Evaluation

Plot return, coverage, policy entropy, state visitation, and regret over time. Compare exploration methods under identical seeds and interaction budgets. In safety-critical applications, track constraint violations and use safe exploration or offline pretraining rather than unrestricted random actions.

10. Complexity and Cost

Epsilon-greedy selection is $O(|A|)$. UCB stores counts/uncertainty per action and is also $O(|A|)$ in a basic bandit. Count-based exploration becomes difficult in continuous/high-dimensional states, requiring density models or learned novelty estimators.

11. Common Use Cases

Online recommendations, clinical trials with safeguards, game exploration, robot skill discovery, ad selection, hyperparameter bandits, and sparse-reward navigation.

12. Common Mistakes

- Decaying epsilon before useful states are discovered
- Leaving epsilon enabled in evaluation
- Treating softmax temperature as scale-independent
- Using random physical actions without safety constraints
- Rewarding raw prediction error in stochastic/noisy regions
- Reporting only final return and hiding exploration cost

13. Edge Cases / Limitations

Random exploration fails in long-horizon sparse tasks. Novelty bonuses may prefer inherently noisy observations. Exploration can harm users or equipment. Non-stationary environments require continued exploration. Offline RL cannot actively explore the environment.

14. Variations

Thompson sampling samples hypotheses; UCB is optimism-based; noisy networks learn parameter noise; count/pseudo-count methods reward novelty; curiosity predicts dynamics; information-gain methods seek uncertainty reduction; safe exploration respects constraints.

15. Related Topics

Bandits isolate exploration without state transitions. **Policy entropy** appears in PPO/A2C. **Offline RL** replaces exploration with dataset coverage concerns. **Model-based RL** can explore in imagination or target uncertain dynamics.

16. Interview Questions

1. **Define exploration and exploitation.** Exploration gathers information; exploitation uses current knowledge for reward.
2. **How does epsilon-greedy work?** Random action with probability epsilon, greedy otherwise.
3. **Why decay epsilon?** Exploration is more valuable when uncertainty is high.
4. **Why can decaying by time be flawed?** Time does not guarantee adequate state-action coverage.
5. **Temperature effect?** High temperature flattens probabilities; low temperature approaches greedy.
6. **What is UCB's idea?** Optimistic value equals estimate plus uncertainty bonus.
7. **What is regret?** Lost cumulative reward relative to an optimal oracle.
8. **Why is exploration harder in RL than bandits?** Actions change future states and information access.
9. **What is the noisy-TV problem?** Curiosity repeatedly favors unpredictable but useless observations.
10. **How is exploration handled offline?** It is not active; algorithms must remain close to supported actions or model uncertainty.

17. Practice Tasks

- Compare epsilon-greedy, UCB, and softmax on bandits.
- Plot regret with confidence bands across seeds.
- Create a non-stationary bandit.
- Debug evaluation contaminated by exploration.
- Add a safe action set to exploratory control.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Bandit strategy lab	Benchmarks exploration regret	NumPy, Plotly	Synthetic bandits	Clear statistical experiment
Curiosity maze agent	Solves sparse navigation	PyTorch, MiniGrid	Procedural mazes	Advanced exploration demo
Safe recommender simulator	Explores under constraints	Pandas, PyTorch	MovieLens simulator	Product/safety relevance

19. Quick Revision

- **Key idea:** learn enough about alternatives while earning reward.
- **Main formula:** $UCB = \hat{\mu} + c\sqrt{\log t / N}$.
- **When to use:** uncertain online decision problems.
- **Metrics:** return, regret, entropy, coverage, violations.
- **Common trap:** premature epsilon decay.
- **Interview one-liner:** "Exploration buys information; exploitation cashes it in."

20. Final Cheat Sheet

Item	Summary
Definition	Information-gathering versus best-known action
Input/output	Values/uncertainty → exploratory action

Item	Summary
Main steps	Estimate, add randomness/bonus, act, update
Hyperparameters	ϵ, τ, c , entropy coefficient
Metrics	Regret, return, coverage, safety
Pros/cons	Finds better behavior / costs reward and risk
Best uses	Online learning with uncertain actions

Bellman Equation

1. Overview

Bellman equations express a value as immediate reward plus discounted value of what follows. They convert a long-horizon return into a one-step recursive relationship and form the foundation of dynamic programming, TD learning, Q-learning, DQN, and actor-critic critics.

2. Intuition

The value of a route equals the reward for the next road segment plus the value of the remaining journey. Solving the whole trip can therefore be reduced to repeatedly combining one-step outcomes with estimates of the future.

3. Prerequisites

MDPs, conditional expectation, returns, policies, state and action values, recursion, fixed points, and basic contraction intuition.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Expectation equation	Value under fixed policy	Policy evaluation	Average over policy actions	V^π, Q^π
Optimality equation	Max over actions	Optimal control	Best next move	Nonlinearity from max
Backup	Apply one-step RHS	Propagates rewards	Update predecessor	DP versus TD
Fixed point	Value unchanged by backup	Defines correct solution	$V = T^\pi V$	Contraction
TD error	Sample Bellman residual	Learning signal	$r + \gamma V' - V$	Semi-gradient

5. Algorithm / Working Process

Choose an equation for policy evaluation or optimal control. Initialize values. Repeatedly back up each state using known expectations, sampled transitions, or minibatches. Stop when values/policy stabilize or the interaction budget ends. Derive a policy from the resulting values.

6. Mathematical Foundation

Bellman expectation for state value:

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s].$$

For action value:

$$Q^\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma \mathbb{E}_{a' \sim \pi} Q^\pi(S_{t+1}, a') \mid s, a].$$

Optimality:

$$V^*(s) = \max_a \mathbb{E}[R_{t+1} + \gamma V^*(S_{t+1}) \mid s, a],$$

$$Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') \mid s, a].$$

For $\gamma < 1$, exact Bellman operators are contractions in finite bounded settings. A Bellman residual measures mismatch between both sides.

7. Practical Implementation

```
import numpy as np

def bellman_policy_backup(P, R, policy, V, gamma=0.99):
    # P: [state, action, next_state], policy: [state, action]
    next_value = np.einsum("sak,k->sa", P, V)
    q = R + gamma * next_value
    return np.sum(policy * q, axis=1)

P = np.array([[[1, 0], [0, 1]], [[0, 1], [1, 0]]], dtype=float)
R = np.array([[0, 1], [0, 0]], dtype=float)
policy = np.full((2, 2), 0.5)
V = np.zeros(2)
for _ in range(100):
    V = bellman_policy_backup(P, R, policy, V, 0.9)
print(V)
```

8. Code Explanation

The expected next value is computed for every state-action pair. Adding reward creates Q-like one-step returns; averaging with the fixed policy produces the new state value. Repetition performs iterative policy evaluation.

9. Training / Evaluation

For exact solvers, track maximum value change and Bellman residual. For learned critics, monitor TD errors, return prediction, value scale, and policy performance. A low empirical residual on a narrow replay buffer does not guarantee accuracy elsewhere.

10. Complexity and Cost

A dense tabular backup over all states/actions/next states costs $O(|S|^2|A|)$. A sampled TD backup is $O(1)$ plus function approximation cost. Exact expectation has low variance but needs a model; sampling scales but introduces noise.

11. Common Use Cases

Policy evaluation, value iteration, Q-learning targets, DQN critic loss, actor-critic TD errors, search leaf backups, and offline value estimation.

12. Common Mistakes

- Mixing Bellman expectation and optimality equations
- Forgetting the expectation over next states
- Forgetting action expectation under π
- Bootstrapping terminal states
- Backpropagating through moving targets inadvertently
- Assuming approximate nonlinear Bellman updates always converge

13. Edge Cases / Limitations

Partial observability violates state sufficiency. $\gamma = 1$ in continuing tasks may produce infinite returns. Function approximation, bootstrapping, and off-policy sampling can diverge. Max operators amplify overestimation.

14. Variations

Finite-horizon Bellman equations include time indices; soft Bellman equations add entropy; distributional equations propagate return distributions; average-reward equations use differential value; constrained variants track reward and cost values.

15. Related Topics

MDPs provide the Markov structure. **Value functions** are Bellman fixed points. **Q-learning/SARSA** use sampled Bellman targets. **DQN** minimizes approximate Bellman error with replay and target networks.

16. Interview Questions

1. **What does a Bellman equation say?** Value equals expected immediate reward plus discounted future value.
2. **Expectation versus optimality equation?** Fixed-policy averaging versus maximization over actions.
3. **What is a Bellman backup?** Applying the right-hand side to update a value estimate.
4. **What is a fixed point?** A value unchanged by its Bellman operator.
5. **Why is discount important for convergence?** It makes standard operators contractions when $\gamma < 1$.
6. **What is TD error?** A sample difference between bootstrapped target and current value.

7. **Why does max cause bias?** It preferentially selects noisy overestimates.
8. **Does zero Bellman residual imply optimality?** Only for the appropriate operator over the relevant full domain and assumptions.
9. **How are MC and Bellman methods different?** MC uses complete sampled returns; Bellman methods bootstrap recursively.
10. **What changes at terminals?** Continuation value is zero.

17. Practice Tasks

- Derive V^π from Q^π .
- Solve a two-state Bellman system analytically.
- Implement policy and optimality backups.
- Visualize convergence for several γ .
- Diagnose a terminal-mask Bellman bug.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Bellman visual lab	Animates value propagation	NumPy, Plotly	GridWorld	Excellent teaching artifact
Exact vs sampled solver	Compares DP and TD	Python, Gymnasium	Random MDPs	Demonstrates theory-to-code
Residual analyzer	Maps critic error by state	PyTorch	CartPole discretization	Debugging-oriented project

19. Quick Revision

- **Key idea:** long-term value decomposes into one step plus future value.
- **Main formula:** $V^\pi(s) = \mathbb{E}[r + \gamma V^\pi(s')]$.
- **When to use:** evaluation, control, and critic learning.
- **Metrics:** Bellman residual, TD error, return.
- **Common trap:** swapping policy expectation for max.
- **Interview one-liner:** "Bellman recursion is the bridge from delayed reward to local learning targets."

20. Final Cheat Sheet

Item	Summary
Definition	Recursive consistency equation for value
Input/output	Current values/model/sample → backed-up values
Main steps	Reward, bootstrap, expectation/max, update
Hyperparameters	γ , backup horizon, step size
Metrics	Residual, TD error, policy return

Item	Summary
Pros/cons	Enables local updates / approximation can diverge
Best uses	Dynamic programming and TD methods

Q-Learning

1. Overview

Q-learning is a model-free, off-policy temporal-difference control algorithm. It learns the optimal action-value function using transitions generated by any sufficiently exploratory behavior policy and a greedy target.

2. Intuition

After taking an action, Q-learning asks: "What reward did I receive, and what is the best value I currently believe I could get from the next state?" It moves the old estimate toward that answer, even if the behavior policy actually chooses something exploratory next.

3. Prerequisites

MDPs, Q-values, Bellman optimality, TD error, epsilon-greedy exploration, learning rates, and tabular Python.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Model-free	No explicit P, R estimate	Learns from samples	Grid experience	Versus dynamic programming
Off-policy	Target differs from behavior	Reuses exploratory data	Epsilon behavior, greedy target	Versus SARSA
TD update	One-step bootstrap	Online and data-efficient	Reward + max next Q	Update equation
GLIE	Infinite exploration, greedy limit	Tabular convergence condition	Decaying epsilon	Assumptions
Max bias	Noisy max overestimates	Can hurt control	Optimistic Q	Double Q

5. Algorithm / Working Process

Initialize $Q(s, a)$, often to zero. For each episode, observe state, choose an exploratory behavior action, step the environment, compute reward plus discounted maximum next Q, update the visited entry, and continue. The learned target policy is greedy with respect to Q. Terminal transitions omit the bootstrap.

6. Mathematical Foundation

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha_t \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

The bracketed quantity is TD error. In a finite tabular MDP, Q-learning converges to Q^* with probability one under standard stochastic-approximation conditions: all state-action pairs are visited infinitely often, rewards are bounded, and step sizes satisfy $\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$.

7. Practical Implementation

```
import numpy as np

def train_q_learning(env, episodes=2_000, alpha=0.1, gamma=0.99, epsilon=0.1):
    q = np.zeros((env.n_states, env.n_actions))
    rng = np.random.default_rng(0)
    for _ in range(episodes):
        state, _ = env.reset()
        done = False
        while not done:
            action = (rng.integers(env.n_actions) if rng.random() < epsilon
                      else rng.choice(np.flatnonzero(q[state] == q[state].max()))
            next_state, reward, terminated, truncated, _ = env.step(int(action))
            target = reward
            if not terminated:
                target += gamma * q[next_state].max()
            q[state, action] += alpha * (target - q[state, action])
            state, done = next_state, terminated or truncated
    return q
```

8. Code Explanation

Random tie-breaking prevents a fixed argmax bias. The behavior policy is epsilon-greedy, while the target always uses the maximum next Q, which makes the method off-policy. True termination stops bootstrapping; truncation ends collection but may retain continuation value.

9. Training / Evaluation

Track rolling episodic return, success rate, state-action visitation, epsilon, TD-error scale, and policy stability. Evaluate a frozen greedy policy over multiple seeds. Tune α , γ , ϵ and epsilon decay; test whether every important state-action pair is explored.

10. Complexity and Cost

Tabular storage is $O(|S||A|)$. Each update is $O(|A|)$ because of next-action maximization. Sample complexity can be high in large spaces. Neural replacement is DQN and adds replay, target networks, and GPU compute.

11. Common Use Cases

Small discrete navigation, games, scheduling, education/interview problems, baseline controllers, and any task with enumerable state-action tables.

12. Common Mistakes

- Calling Q-learning on-policy because behavior is epsilon-greedy
- Using the chosen next action instead of max and still naming it Q-learning
- No exploration or overly fast epsilon decay
- Bootstrapping true terminal states
- Fixed argmax tie bias
- Expecting tabular convergence guarantees with a nonlinear network

13. Edge Cases / Limitations

Tables fail for continuous or huge state spaces. Random exploration is weak for sparse long-horizon rewards. Q-learning can be unsafe because its target assumes greedy continuation while behavior explores. Function approximation can diverge without stabilizers.

14. Variations

Double Q-learning reduces overestimation. Expected Q-learning averages under a target policy. n-step Q-learning propagates rewards faster. Watkins's $Q(\lambda)$ uses eligibility traces. DQN applies neural approximation.

15. Related Topics

SARSA is on-policy and uses the actual next action. **Bellman optimality** supplies Q-learning's max target. **DQN** scales the idea to high-dimensional states. **Offline RL** modifies Q-learning to handle unsupported actions.

16. Interview Questions

1. **Is Q-learning on-policy?** No, its greedy target differs from exploratory behavior.
2. **Write the update.** $Q \leftarrow Q + \alpha[r + \gamma \max Q' - Q]$.
3. **Why model-free?** It never explicitly estimates transition probabilities.
4. **What policy does it learn?** The greedy optimal policy under convergence assumptions.
5. **Why explore if target is greedy?** Without coverage, Q-values for untried actions cannot be learned.
6. **Q-learning versus value iteration?** Sampled asynchronous backups versus model-based full expectations.
7. **What causes overestimation?** Max over noisy Q estimates.
8. **What are convergence assumptions?** Finite tabular MDP, bounded rewards, sufficient visits, suitable step sizes.
9. **How treat terminal transitions?** Target equals immediate reward.
10. **Why is random tie-breaking useful?** It avoids systematic preference among equal estimates.

17. Practice Tasks

- Implement tabular Q-learning on FrozenLake.
- Compare constant and decaying epsilon.
- Add random argmax tie-breaking.
- Demonstrate terminal-bootstrap failure.
- Implement Double Q-learning and compare Q scale.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
FrozenLake study	Compares exploration schedules	NumPy, Gymnasium	FrozenLake	Clean reproducible baseline
Dynamic routing agent	Learns paths with stochastic costs	Python, NetworkX	Synthetic networks	Algorithms/control crossover
Double-Q dashboard	Shows bias and policy effects	NumPy, Plotly	Random MDP suite	Interview-ready analysis

19. Quick Revision

- **Key idea:** learn optimal Q off-policy from sampled TD targets.
- **Main formula:** $r + \gamma \max_{a'} Q(s', a')$.
- **When to use:** finite discrete model-free control.
- **Metrics:** return, success, coverage, TD error.
- **Common trap:** confusing max target with SARSA's next-action target.
- **Interview one-liner:** "Q-learning behaves exploratorily but learns about greedy continuation."

20. Final Cheat Sheet

Item	Summary
Definition	Off-policy TD control
Input/output	Transition $\rightarrow$ updated Q table
Main steps	Explore, step, max bootstrap, update
Hyperparameters	α, γ, ϵ
Metrics	Return, TD error, coverage
Pros/cons	Simple and model-free / tables and max bias
Best uses	Small discrete MDPs

SARSA

1. Overview

SARSA is an on-policy temporal-difference control algorithm named after the transition sequence State–Action–Reward–State–Action. It learns the Q-value of the same policy used to collect data, including that policy's exploration.

2. Intuition

Q-learning evaluates the safest outcome assuming perfect greedy behavior after the next state. SARSA evaluates what the exploratory agent will actually do. Near a cliff, SARSA may learn a wider safe path because it accounts for the chance of a future random exploratory step.

3. Prerequisites

Q-values, TD learning, epsilon-greedy behavior, Q-learning, on-policy versus off-policy learning, and terminal handling.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
On-policy target	Uses behavior's next action	Learns exploratory policy value	Epsilon-greedy continuation	Versus Q-learning
Five-tuple	S, A, R, S', A'	Names algorithm	One transition plus next choice	Update order
Policy improvement	Behavior becomes greedy gradually	Converges under GLIE	Epsilon decay	Convergence
Risk during training	Exploration included in learned values	Can favor safer routes	Cliff Walking	Canonical comparison
Expected SARSA	Averages over next policy	Lower target variance	Weighted next Q	Variation

5. Algorithm / Working Process

Initialize Q. At episode start choose A from the current behavior policy. Step with A , observe R, S' , choose A' from the same current policy, update Q toward $R + \gamma Q(S', A')$, and shift $S \leftarrow S', A \leftarrow A'$. At a terminal transition use only R .

6. Mathematical Foundation

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)],$$

where $A_{t+1} \sim \pi(\cdot | S_{t+1})$, the same policy generating data. Expected SARSA replaces the sampled next value with

$$\sum_a \pi(a | S_{t+1}) Q(S_{t+1}, a),$$

reducing sampling variance while retaining an on-policy target.

7. Practical Implementation

```

import numpy as np

def epsilon_greedy(q_row, epsilon, rng):
    if rng.random() < epsilon:
        return int(rng.integers(len(q_row)))
    return int(rng.choice(np.flatnonzero(q_row == q_row.max()))))

def train_sarsa(env, episodes=2_000, alpha=0.1, gamma=0.99, epsilon=0.1):
    q = np.zeros((env.n_states, env.n_actions))
    rng = np.random.default_rng(0)
    for _ in range(episodes):
        state, _ = env.reset()
        action = epsilon_greedy(q[state], epsilon, rng)
        while True:
            next_state, reward, terminated, truncated, _ = env.step(action)
            target = reward
            if not terminated:
                next_action = epsilon_greedy(q[next_state], epsilon, rng)
                target += gamma * q[next_state, next_action]
            q[state, action] += alpha * (target - q[state, action])
            if terminated or truncated:
                break
            state, action = next_state, next_action
    return q

```

8. Code Explanation

The next action is selected before the update and then reused on the next loop iteration. This detail makes the update genuinely on-policy. If max next Q replaced the selected next action, the code would implement Q-learning.

9. Training / Evaluation

Report training return under exploration and evaluation return under a frozen greedy policy; these answer different questions. Track cliff/failure frequency, epsilon, TD error, and state-action coverage. Compare SARSA with Q-learning using identical seeds and behavior schedules.

10. Complexity and Cost

Storage is $O(|S||A|)$; action selection is $O(|A|)$; each update is $O(1)$ after choosing the next action. Like tabular Q-learning, sample complexity grows poorly with state space.

11. Common Use Cases

Small discrete control tasks, online control where exploratory behavior matters, risk-sensitive teaching examples, and baselines for on-policy TD methods.

12. Common Mistakes

- Sampling a next action for the target and resampling another for execution
- Replacing $Q(s', a')$ by max Q
- Claiming SARSA learns the greedy policy while epsilon remains fixed
- Bootstrapping terminals
- Comparing training risk with greedy evaluation risk
- Decaying epsilon without sufficient exploration

13. Edge Cases / Limitations

Fixed epsilon means SARSA evaluates a permanently exploratory policy, which may be suboptimal at deployment. Tables do not scale. Sampled next actions add variance. On-policy data reuse is limited compared with off-policy methods.

14. Variations

Expected SARSA averages next values under the policy. n-step SARSA uses multiple rewards. SARSA(λ) uses eligibility traces. Differential SARSA targets average reward in continuing tasks. Deep SARSA uses function approximation but loses simple tabular guarantees.

15. Related Topics

Q-learning is off-policy with a max target. **A2C** also learns on-policy but uses a parameterized actor and critic. **Eligibility traces** bridge one-step TD and Monte Carlo.

16. Interview Questions

1. **Why is it called SARSA?** Its update uses $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$.
2. **Is SARSA on-policy?** Yes, target and data use the same behavior policy.
3. **Write its target.** $r + \gamma Q(s', a')$, where a' is selected by the current policy.
4. **SARSA versus Q-learning near a cliff?** SARSA accounts for exploratory risk and often learns a safer route.
5. **What if epsilon remains positive?** It learns the value of an epsilon-greedy, not purely greedy, policy.
6. **What is Expected SARSA?** It averages next Q under the policy rather than sampling one next action.
7. **Why can Expected SARSA be lower variance?** It removes randomness from next-action sampling.
8. **How are terminals handled?** No next action or bootstrap is used.
9. **Can SARSA learn off replay data?** Standard SARSA is on-policy; arbitrary old replay creates policy mismatch.
10. **What does GLIE mean?** Greedy in the limit with infinite exploration.

17. Practice Tasks

- Implement SARSA on CliffWalking.
- Compare paths with Q-learning.
- Add Expected SARSA.
- Test fixed versus decayed epsilon.
- Debug a resampled-next-action bug.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Cliff safety study	Compares on/off-policy risk	NumPy, Gymnasium	CliffWalking	Classic interview demonstration
Expected SARSA lab	Measures target variance	Python, Plotly	Windy GridWorld	Strong experimental reasoning
Online routing controller	Adapts while acting	Python, NetworkX	Stochastic graph	Real-time control framing

19. Quick Revision

- **Key idea:** learn Q for the policy actually being followed.
- **Main formula:** $r + \gamma Q(s', a')$.
- **When to use:** on-policy discrete TD control.
- **Metrics:** training/evaluation return, risk, TD error.
- **Common trap:** selecting a' twice.
- **Interview one-liner:** "SARSA learns the consequences of its exploratory behavior, not an imaginary greedy continuation."

20. Final Cheat Sheet

Item	Summary
Definition	On-policy TD control
Input/output	SARSA tuple $\rightarrow$ updated Q
Main steps	Choose action, step, choose next, update
Hyperparameters	α, γ, ϵ
Metrics	Return, failures, TD error
Pros/cons	Accounts for behavior risk / on-policy and tabular limits
Best uses	Small online discrete tasks

Deep Q-Network

1. Overview

A Deep Q-Network (DQN) approximates action-values with a neural network, typically taking a state as input and outputting one Q -value per discrete action. Experience replay, a target network, and careful optimization stabilize neural Q -learning.

2. Intuition

A Q -table cannot store a row for every possible image. DQN learns shared visual features so similar situations receive related values. Replay shuffles past experience, while the target network gives the learner a temporarily

stable objective.

3. Prerequisites

Q-learning, Bellman optimality, neural networks, PyTorch, minibatch SGD, replay buffers, convolutional networks, and exploration.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Q-network	$Q_{\theta}(s, a)$ approximator	Generalizes across states	CNN for Atari	Output dimension
Replay buffer	Stores transitions	Breaks correlation and reuses data	Random minibatches	Off-policy compatibility
Target network	Delayed copy $Q_{\bar{\theta}}$	Stabilizes moving targets	Periodic sync	Why needed
Huber loss	Robust TD regression	Limits outlier gradients	Smooth L1	MSE versus Huber
Epsilon-greedy	Behavior exploration	Collects varied data	Epsilon schedule	Train/eval

5. Algorithm / Working Process

Initialize online and target networks, copy weights, and create replay memory. Interact using epsilon-greedy online Q-values. Store transitions. After warm-up, sample random batches, compute $y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')$, regress selected online Q-values toward y , clip gradients if needed, and periodically update the target network.

Input is a batch of state tensors. Output is an $|A|$ -vector per state. Training gathers transitions and backpropagates TD loss; inference takes argmax.

6. Mathematical Foundation

$$y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\theta}}(s'_i, a'),$$

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_i \text{Huber}(Q_{\theta}(s_i, a_i) - y_i).$$

The target is semi-gradient: y_i is detached. Replay changes the sampling distribution but Q-learning is off-policy. The “deadly triad”—function approximation, bootstrapping, and off-policy learning—explains why stabilizers are important.

7. Practical Implementation

```

import torch
from torch import nn
from torch.nn import functional as F

class DQN(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_actions)
        )

    def forward(self, x):
        return self.net(x)

def dqn_loss(online, target, batch, gamma=0.99):
    states, actions, rewards, next_states, terminated = batch
    chosen_q = online(states).gather(1, actions[:, None]).squeeze(1)
    with torch.no_grad():
        next_q = target(next_states).max(dim=1).values
        targets = rewards + gamma * (1 - terminated.float()) * next_q
    return F.smooth_l1_loss(chosen_q, targets)

online, target = DQN(4, 2), DQN(4, 2)
target.load_state_dict(online.state_dict())

```

8. Code Explanation

The network emits all action values. "gather" selects the value of each executed action. The target network is evaluated without gradients. Smooth L1 is Huber loss. A full training loop additionally needs environment collection, replay sampling, optimizer steps, epsilon scheduling, and periodic target synchronization.

9. Training / Evaluation

Warm the replay buffer before learning. Normalize observations, clip or scale rewards only deliberately, and evaluate frozen greedy checkpoints. Track episodic return, success, replay coverage, epsilon, TD loss, Q magnitudes, gradient norms, and target-update effects. Use several seeds because DQN variance is high.

10. Complexity and Cost

Replay memory is $O(N(\text{state bytes} + \text{transition metadata}))$. Each update performs online and target forward passes plus one backward pass. Inference is one network pass and argmax $O(|A|)$. Pixel DQN generally benefits from GPU; low-dimensional tasks run on CPU.

11. Common Use Cases

Atari games, discrete robot actions, navigation, scheduling, traffic signals, and discrete resource allocation.

12. Common Mistakes

- Using the online network for both rapidly changing prediction and target without stabilization
- Forgetting “gather” and training every action toward one target
- Bootstrapping terminal transitions
- Not detaching target values
- Starting updates before replay has variety
- Applying DQN directly to continuous actions
- Reporting one lucky seed

13. Edge Cases / Limitations

DQN is unsuitable for large or continuous action spaces without modification. Replay can contain stale data in non-stationary environments. Q-values overestimate, extrapolate poorly, and can diverge. Sparse rewards and partial observations still require better exploration or memory.

14. Variations

Double DQN separates selection and evaluation; Dueling DQN decomposes value and advantage; prioritized replay samples large-error transitions; n-step DQN propagates rewards; distributional DQN models return distributions; Rainbow combines several extensions.

15. Related Topics

Q-learning is its tabular foundation. **Double Q-learning** reduces max bias. **Offline RL** must control replay-distribution extrapolation. **Actor-critic** handles continuous actions with a learned actor.

16. Interview Questions

1. **What does DQN output?** One estimated Q-value per discrete action.
2. **Why replay?** It reuses experience and reduces temporal correlation in minibatches.
3. **Why a target network?** It slows target drift and stabilizes bootstrapping.
4. **Why is DQN off-policy?** Replay behavior may differ from the greedy target policy.
5. **What is the DQN target?** $r + \gamma(1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')$.
6. **Why use Huber loss?** It is quadratic near zero and less sensitive to large TD errors.
7. **How does Double DQN help?** Online network selects action; target network evaluates it.
8. **Why not continuous actions?** Computing max over infinitely many actions is hard.
9. **What is the deadly triad?** Off-policy learning, bootstrapping, and function approximation.
10. **Hard versus soft target update?** Periodic full copy versus Polyak averaging.

17. Practice Tasks

- Train DQN on CartPole.
- Remove replay or target network and compare stability.
- Implement Double DQN.
- Visualize replay action/reward coverage.
- Debug incorrect terminal masks and action gathers.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Atari-lite DQN	Learns from stacked pixels	PyTorch, Gymnasium	ALE/Pong	Canonical deep RL project
Traffic-light DQN	Controls discrete signals	PyTorch, SUMO	Traffic simulator	Applied control evidence
Rainbow ablation	Compares DQN extensions	PyTorch	LunarLander	Research-style ablation

19. Quick Revision

- **Key idea:** neural Q-learning stabilized by replay and target network.
- **Main formula:** $\text{Huber}(Q_\theta(s, a), r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a'))$.
- **When to use:** high-dimensional states, small discrete actions.
- **Metrics:** return, TD loss, Q scale, replay coverage.
- **Common trap:** forgetting target detachment or terminal mask.
- **Interview one-liner:** "DQN is Q-learning plus function approximation, replay, and a slowly moving target."

20. Final Cheat Sheet

Item	Summary
Definition	Neural discrete-action Q-learning
Input/output	State tensor $\rightarrow$ Q for each action
Main steps	Explore, store, sample, bootstrap, optimize, sync
Hyperparameters	LR, γ , ϵ , replay size, batch, target interval
Metrics	Return, TD loss, Q magnitude
Pros/cons	Handles rich states / unstable and discrete-only
Best uses	Pixel or vector observations with discrete actions

Policy Gradient

1. Overview

Policy-gradient methods directly optimize parameters of a stochastic policy to increase expected return. They avoid explicit action maximization, naturally handle continuous actions, and underpin REINFORCE, actor-critic, A2C/A3C, PPO, and much of RLHF.

2. Intuition

If an action leads to a better-than-expected result, make it more likely in similar states; if it leads to a worse result, make it less likely. The log-probability trick converts this intuitive probability adjustment into a

differentiable objective.

3. Prerequisites

Calculus, gradients, probability densities, log-likelihood, expectation, Monte Carlo returns, neural policies, and variance reduction.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Score function	$\nabla \log \pi_{\theta}(a \mid s)$	Differentiates sampled actions	Categorical log-prob	Log-derivative trick
Return weighting	Reinforce successful actions	Supplies learning direction	High-return trajectory	Credit assignment
Baseline	Subtract action-independent estimate	Reduces variance without bias	$G_t - V(s)$	Why valid
Entropy bonus	Encourages stochasticity	Preserves exploration	Add $+\beta H$	Sign convention
On-policy sampling	Data from current policy	Keeps gradient valid	Fresh rollouts	Sample inefficiency

5. Algorithm / Working Process

Collect trajectories with the current policy, compute reward-to-go for each action, optionally normalize returns and subtract a baseline, form loss as negative log-probability times advantage, add entropy regularization, backpropagate, and discard or carefully limit reuse of old data.

6. Mathematical Foundation

Using the likelihood-ratio identity:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) G_t \right].$$

With baseline $b(s)$:

$$\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(A_t \mid S_t) b(S_t)] = 0.$$

The sampled loss minimized in code is

$$\mathcal{L}_{\text{PG}} = -\frac{1}{N} \sum_t \log \pi_{\theta}(a_t \mid s_t) \hat{A}_t - \beta H(\pi_{\theta}(\cdot \mid s_t)).$$

REINFORCE is unbiased under correct sampling but can have high variance.

7. Practical Implementation

```
import torch

def reinforce_loss(log_probs, rewards, gamma=0.99):
    returns, running = [], 0.0
    for reward in reversed(rewards):
        running = reward + gamma * running
        returns.append(running)
    returns = torch.tensor(list(reversed(returns)), dtype=torch.float32)
    returns = (returns - returns.mean()) / (returns.std() + 1e-8)
    return -(torch.stack(log_probs) * returns).sum()

# Training usage:
# loss = reinforce_loss(saved_log_probs, episode_rewards)
# optimizer.zero_grad(); loss.backward(); optimizer.step()
```

8. Code Explanation

Reward-to-go is computed backward. Normalization reduces scale variability but does not replace a learned state baseline. Multiplying log-probability by positive return increases that action's likelihood under gradient descent on the negative objective.

9. Training / Evaluation

Normalize advantages, monitor entropy and KL, clip gradients, and aggregate enough trajectories for stable estimates. Evaluate frozen checkpoints over multiple seeds. Metrics include return, success, entropy, policy loss, gradient norm, KL per update, and sample efficiency.

10. Complexity and Cost

Each rollout step costs one policy forward pass; each update costs a backward pass over collected steps. Memory is $O(T)$ for stored trajectory tensors. Vanilla on-policy methods discard data after one update, so environment sample cost is often the bottleneck.

11. Common Use Cases

Continuous robot control, stochastic policies, games, sequence generation, neural architecture/control decisions, and LLM post-training.

12. Common Mistakes

- Using full episode return for every action instead of reward-to-go without understanding variance
- Forgetting the negative sign in a minimized loss
- Backpropagating through sampled returns
- Reusing stale on-policy data freely
- Computing log-probability of a different action than executed
- Omitting baselines and then blaming unstable optimization

13. Edge Cases / Limitations

Gradients have high variance, sparse rewards remain difficult, and large policy updates can collapse behavior. On-policy sampling is expensive. Gaussian policies can struggle with bounds and multimodality. Reward misspecification is directly optimized and may be exploited.

14. Variations

REINFORCE uses Monte Carlo returns. Natural policy gradient preconditions by Fisher information. TRPO constrains KL. PPO uses a clipped surrogate. Deterministic policy gradients differentiate Q through continuous actions. Actor-critic learns a baseline.

15. Related Topics

Advantage functions weight log-probability gradients. **Actor-critic** supplies a learned critic. **PPO** controls update size. **RLHF** applies policy-gradient-style optimization to language-model policies and learned rewards.

16. Interview Questions

1. **Why direct policy optimization?** It handles stochastic and continuous actions without argmax over Q.
2. **State the policy-gradient theorem.** Expected score-function gradient weighted by Q or advantage.
3. **Why log probability?** $\nabla \pi = \pi \nabla \log \pi$ enables gradients through sampled distributions.
4. **What does a positive advantage do?** Increases probability of the sampled action.
5. **Why can a baseline be subtracted?** An action-independent baseline has zero expected score-function contribution.
6. **What is REINFORCE?** Monte Carlo policy gradient using sampled returns.
7. **Why high variance?** Outcomes depend on many stochastic future actions and transitions.
8. **Why on-policy?** The expectation is under the current policy's trajectory distribution.
9. **Role of entropy?** Encourages exploration and prevents premature collapse.
10. **Policy-gradient versus DQN?** Direct policy optimization and continuous support versus value-based discrete off-policy learning.

17. Practice Tasks

- Implement REINFORCE on CartPole.
- Compare total return and reward-to-go weighting.
- Add a moving-average baseline.
- Plot entropy and KL during training.
- Debug a detached log-probability error.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
REINFORCE from scratch	Solves classic control	PyTorch, Gymnasium	CartPole	Demonstrates mathematical implementation
Continuous policy lab	Compares Gaussian parameterizations	PyTorch	Pendulum	Strong action-distribution insight

Project	What it does	Tech stack	Dataset/environment	Resume value
Sequence optimizer	Learns token sequences from reward	PyTorch	Synthetic grammar	LLM/RL bridge

19. Quick Revision

- **Key idea:** increase likelihood of actions with positive estimated advantage.
- **Main formula:** $\nabla J = \mathbb{E}[\nabla \log \pi \hat{A}]$.
- **When to use:** stochastic/continuous differentiable policies.
- **Metrics:** return, entropy, KL, gradient variance.
- **Common trap:** wrong loss sign or stale data reuse.
- **Interview one-liner:** "Policy gradients turn sampled rewards into likelihood updates without differentiating through the environment."

20. Final Cheat Sheet

Item	Summary
Definition	Direct gradient ascent on expected policy return
Input/output	Trajectories $\rightarrow$ updated policy parameters
Main steps	Roll out, estimate return/advantage, log-prob loss, update
Hyperparameters	LR, γ , entropy, batch horizon
Metrics	Return, entropy, KL, gradients
Pros/cons	Continuous/stochastic control / high variance and on-policy cost
Best uses	Neural policy optimization

Actor-Critic

1. Overview

Actor-critic methods learn both a policy (actor) and a value estimator (critic). The critic estimates how good states or actions are; the actor uses that estimate to update behavior. This combines policy-gradient flexibility with lower-variance bootstrapped learning.

2. Intuition

The actor is a performer and the critic is a coach. The performer chooses an action; the coach says whether it was better or worse than expected. The performer adjusts toward actions receiving positive feedback.

3. Prerequisites

Policy gradients, value/Q functions, TD error, neural networks, bootstrapping, and bias-variance trade-offs.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Actor	Parameterized policy π_θ	Selects actions	Categorical/Gaussian network	Actor loss
Critic	V_ϕ or Q_ϕ	Evaluates behavior	Predict return	Critic target
TD error	$r + \gamma V' - V$	One-step advantage estimate	Positive surprise	Bias/variance
Shared encoder	Features used by actor/critic	Saves compute	Common CNN trunk	Gradient interference
On/off-policy	Data-policy relationship	Defines corrections and reuse	A2C versus SAC	Algorithm families

5. Algorithm / Working Process

Roll out the actor, store states/actions/rewards/log-probabilities/values, construct TD or multi-step targets, update the critic to predict returns, update the actor using detached advantages, add entropy regularization, and repeat. At inference only the actor is needed unless the critic supports planning or diagnostics.

6. Mathematical Foundation

For a state-value critic:

$$\delta_t = R_{t+1} + \gamma(1 - d_t)V_\phi(S_{t+1}) - V_\phi(S_t).$$

Actor loss:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}[\log \pi_\theta(A_t | S_t) \text{stopgrad}(\hat{A}_t)] - \beta \mathbb{E}[H(\pi_\theta)].$$

Critic loss:

$$\mathcal{L}_{\text{critic}} = \frac{1}{2} \mathbb{E}[(V_\phi(S_t) - \hat{G}_t)^2].$$

One-step δ_t is a conditionally unbiased estimate of the advantage when $V_\phi = V^\pi$.

7. Practical Implementation

```
import torch
from torch import nn

class ActorCritic(nn.Module):
    def __init__(self, state_dim, n_actions):
        super().__init__()
```

```

self.body = nn.Sequential(nn.Linear(state_dim, 128), nn.Tanh())
self.actor = nn.Linear(128, n_actions)
self.critic = nn.Linear(128, 1)

def forward(self, states):
    features = self.body(states)
    return self.actor(features), self.critic(features).squeeze(-1)

def losses(model, states, actions, rewards, next_states, terminated, gamma=0.99):
    logits, values = model(states)
    dist = torch.distributions.Categorical(logits=logits)
    with torch.no_grad():
        _, next_values = model(next_states)
        targets = rewards + gamma * (1 - terminated.float()) * next_values
        advantages = targets - values
    actor_loss = -(dist.log_prob(actions) * advantages).mean() - 0.01 *
    dist.entropy().mean()
    critic_loss = 0.5 * (values - targets).pow(2).mean()
    return actor_loss, critic_loss

```

8. Code Explanation

A shared body feeds separate policy-logit and scalar-value heads. The critic constructs a one-step target. Advantages are detached so actor gradients do not manipulate the critic to make actions appear good. Actor and critic losses can be combined with a value-loss coefficient.

9. Training / Evaluation

Track return, entropy, actor loss, value loss, explained variance, KL, advantage mean/std, and gradient norms. Balance actor and critic learning rates; an underfit critic adds variance, while an overfit or biased critic misdirects the actor. Evaluate without updates over multiple seeds.

10. Complexity and Cost

Each action requires actor inference; a shared actor-critic adds only a small value head. Training requires backward passes for both objectives and rollout storage. On-policy actor-critics are sample-hungry; off-policy variants add replay buffers and target critics.

11. Common Use Cases

Discrete and continuous control, robotics, games, navigation, simulation optimization, recommendation, and large-model policy optimization.

12. Common Mistakes

- Not detaching advantage in the actor loss
- Bootstrapping terminal transitions
- Allowing critic loss scale to dominate shared features
- Using stale log-probabilities without correction
- Assuming critic loss alone reflects policy quality

- Mixing Q-critic and V-critic formulas

13. Edge Cases / Limitations

Biased critic estimates create biased policy gradients. Shared representations can cause gradient conflict. On-policy variants waste experience; off-policy variants require corrections and stability devices. Sparse reward still challenges both networks.

14. Variations

A2C/A3C use on-policy V-critics; PPO adds clipped policy updates; DDPG/TD3 use deterministic actors and Q-critics; SAC uses stochastic actors, twin critics, replay, and entropy; IMPALA corrects distributed behavior lag.

15. Related Topics

Advantage is the actor's learning signal. **PPO** is a stable actor-critic method. **A2C/A3C** differ in rollout synchronization. **Q-values** are used by continuous off-policy critics.

16. Interview Questions

1. **What are actor and critic?** Policy and value estimator.
2. **Why add a critic?** It supplies a low-variance learned baseline/advantage.
3. **What is the actor loss?** Negative log-probability weighted by detached advantage, often plus entropy.
4. **What is the critic loss?** Regression to bootstrapped or multi-step return targets.
5. **Why detach advantage?** Actor update should change policy, not alter its evaluation signal.
6. **Can actor-critic be off-policy?** Yes—DDPG, TD3, and SAC are examples.
7. **V-critic versus Q-critic?** V evaluates state; Q evaluates state-action and may guide continuous actors.
8. **What if critic is poor?** Actor gradients become noisy or systematically biased.
9. **Why use separate learning rates?** Actor and critic objectives have different scales and dynamics.
10. **Is the critic needed at inference?** Often no; the actor alone selects actions.

17. Practice Tasks

- Implement one-step actor-critic.
- Add n-step returns and compare stability.
- Measure value explained variance.
- Test separate versus shared encoders.
- Debug an accidental advantage-gradient path.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Actor-critic control	Solves CartPole/Pendulum	PyTorch, Gymnasium	Classic Control	Core deep RL competence

Project	What it does	Tech stack	Dataset/environment	Resume value
Bias-variance study	Compares 1-step and n-step targets	PyTorch, Plotly	LunarLander	Research-style analysis
Shared-feature ablation	Studies gradient interference	PyTorch	MiniGrid	Architecture/evaluation strength

19. Quick Revision

- **Key idea:** actor acts; critic estimates whether the action was better than expected.
- **Main formula:** actor uses $-\log \pi(a | s) \hat{A}$; critic fits return.
- **When to use:** scalable neural policy optimization.
- **Metrics:** return, entropy, KL, explained variance.
- **Common trap:** failing to detach advantages.
- **Interview one-liner:** "Actor-critic trades some critic bias for much lower policy-gradient variance."

20. Final Cheat Sheet

Item	Summary
Definition	Joint policy and value learning
Input/output	State $\rightarrow$ action distribution and value
Main steps	Roll out, target, critic update, actor update
Hyperparameters	Actor/critic LR, γ , entropy, value coefficient
Metrics	Return, entropy, value loss/explained variance
Pros/cons	Flexible and lower variance / coupled instability
Best uses	Modern discrete and continuous deep RL

Advantage Function

1. Overview

The advantage function measures how much better an action is than the policy's average action at the same state:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

It is the preferred learning signal for many policy-gradient algorithms because it centers action quality around a state-dependent baseline.

2. Intuition

Scoring 80% is impressive in a difficult exam where the expected score is 50%, but not in an easy exam where the expected score is 95%. Q is the raw score; V is the local expectation; advantage is performance relative to that context.

3. Prerequisites

State values, Q-values, policy gradients, TD error, actor-critic methods, expectations, and bias-variance trade-offs.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Exact advantage	$Q^\pi - V^\pi$	Relative action quality	Action is +2 above baseline	Definition
Zero policy mean	$\mathbb{E}_{a \sim \pi} A^\pi = 0$	Centers updates	Positives balance negatives	Derivation
TD advantage	One-step TD error	Cheap estimate	$r + \gamma V' - V$	When valid
GAE	Weighted multi-step TD errors	Bias-variance control	λ trace	PPO implementation
Normalization	Batch center/scale	Improves optimizer conditioning	Mean zero, std one	Does not fix bias

5. Algorithm / Working Process

Collect rewards and value predictions. Compute TD residuals backward. Combine residuals using Generalized Advantage Estimation (GAE), reset recursion at true terminals, optionally normalize the resulting batch, and use detached advantages in the actor objective. Separately compute return targets for critic training.

6. Mathematical Foundation

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s), \quad \sum_a \pi(a | s) A^\pi(s, a) = 0.$$

One-step estimate:

$$\delta_t = r_t + \gamma(1 - d_t)V(s_{t+1}) - V(s_t).$$

GAE:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l},$$

computed recursively as $\hat{A}_t = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}$. Low λ has more bootstrap bias and lower variance; $\lambda \rightarrow 1$ approaches Monte Carlo behavior.

7. Practical Implementation

```
import torch

def generalized_advantage(rewards, values, next_value, terminated, gamma=0.99,
lam=0.95):
    advantages = torch.zeros_like(rewards)
    gae = torch.tensor(0.0, device=rewards.device)
    for t in reversed(range(len(rewards))):
        continuation = 1.0 - terminated[t].float()
        following_value = next_value if t == len(rewards) - 1 else values[t + 1]
        delta = rewards[t] + gamma * continuation * following_value - values[t]
        gae = delta + gamma * lam * continuation * gae
        advantages[t] = gae
    returns = advantages + values
    return advantages, returns
```

8. Code Explanation

The loop runs backward because each advantage depends on later residuals. The terminal mask stops both value bootstrapping and GAE carryover. "advantages + values" produces critic return targets. In practice values and returned tensors are detached before actor optimization.

9. Training / Evaluation

Track advantage mean, standard deviation, extremes, correlation with realized returns, critic explained variance, and resulting KL updates. Normalize per training batch, not across unrelated evaluation data. Verify boundaries carefully when rollouts contain fragments from multiple environments.

10. Complexity and Cost

GAE costs $O(T)$ time and $O(T)$ storage for a rollout. It adds negligible compute relative to neural forward/backward passes. Longer rollout horizons increase memory but can improve return estimation.

11. Common Use Cases

PPO, A2C/A3C, TRPO, REINFORCE with learned baseline, policy diagnostics, and dueling-network decomposition.

12. Common Mistakes

- Reversing the sign $V - Q$
- Allowing actor gradients through the critic/advantage
- Carrying GAE across episode boundaries
- Treating truncation exactly like termination automatically
- Normalizing one sample or tiny batches
- Believing normalized advantages are unbiased merely because mean is zero

13. Edge Cases / Limitations

Poor value estimates bias or add variance to advantages. Very sparse reward can make most advantages uninformative. A scalar advantage hides risk. Truncated recurrent sequences may need bootstrap values and hidden-state care.

14. Variations

One-step TD, n-step advantage, Monte Carlo advantage, GAE, V-trace, Retrace, and distributional/risk-sensitive advantages differ in return estimator and off-policy correction. GAE is the placement and implementation priority.

15. Related Topics

Q-value is absolute action return; **V-value** is the policy baseline. **Actor-critic** estimates both policy and baseline. **PPO** uses GAE-weighted probability ratios. **Dueling DQN** represents Q using value and centered advantage.

16. Interview Questions

1. **Define advantage.** $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$.
2. **Why use it in policy gradients?** It centers credit by state and reduces variance.
3. **Expected advantage under policy?** Zero at each state.
4. **How can TD error estimate advantage?** If the value function is correct, its conditional expectation equals advantage.
5. **What is GAE?** An exponentially weighted sum of future TD residuals.
6. **Role of λ ?** Controls bootstrap bias versus sampling variance.
7. **Why normalize advantages?** Better numerical conditioning and stable step scale.
8. **Should gradients flow through advantages?** Not in the standard actor update.
9. **Advantage versus return?** Return is trajectory outcome; advantage compares an action against state baseline.
10. **How handle terminals in GAE?** Zero continuation and stop recursive carryover.

17. Practice Tasks

- Verify policy-weighted advantages sum to zero.
- Implement GAE and compare with explicit sums.
- Plot results for several λ .
- Debug GAE across concatenated episodes.
- Measure update variance with return versus advantage.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
GAE explorer	Visualizes bias/variance by lambda	NumPy, Plotly	Synthetic trajectories	Strong math communication
Advantage debugger	Inspects PPO rollout statistics	PyTorch, Gymnasium	LunarLander	Practical debugging signal

Project	What it does	Tech stack	Dataset/environment	Resume value
Dueling value viewer	Separates state/action effects	PyTorch	Atari-lite	Connects value architectures

19. Quick Revision

- **Key idea:** relative action value after subtracting state baseline.
- **Main formula:** $A = Q - V$; $GAE = \sum (\gamma \lambda)^l \delta_{t+l}$.
- **When to use:** variance-reduced actor updates.
- **Metrics:** mean/std, correlation, critic quality.
- **Common trap:** leaking gradients through advantage.
- **Interview one-liner:** "Advantage asks whether this action was better than what was normally expected here."

20. Final Cheat Sheet

Item	Summary
Definition	Relative state-action value
Input/output	Rewards and values → per-step learning weights
Main steps	TD residuals, backward GAE, normalize, detach
Hyperparameters	γ, λ
Metrics	Advantage stats, explained variance
Pros/cons	Lower variance / depends on critic quality
Best uses	PPO/A2C policy updates

PPO

1. Overview

Proximal Policy Optimization (PPO) is an on-policy actor-critic algorithm that limits destructive policy changes using a clipped probability-ratio objective. It is popular because it is comparatively stable, straightforward to implement, works with discrete and continuous actions, and scales to robotics, games, and RLHF.

2. Intuition

After collecting experience, we want the policy to favor good actions—but not change so much that the old experience becomes irrelevant. PPO allows improvement inside a trust-like region and removes incentive for excessively large probability changes.

3. Prerequisites

Policy gradients, actor-critic, importance ratios, advantage/GAE, entropy, KL divergence, minibatch optimization, and PyTorch distributions.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Old policy	Policy that collected rollout	Defines denominator	Saved log-probs	Must freeze
Ratio	$r_t = \pi_\theta / \pi_{\text{old}}$	Measures policy change	$\exp(\text{new-old log-prob})$	Importance sampling
Clipped surrogate	Limits useful ratio range	Reduces destructive updates	$[1 - \epsilon, 1 + \epsilon]$	min behavior
GAE	Advantage estimator	Stable learning signal	$\lambda = 0.95$	Bias-variance
Multiple epochs	Reuses rollout modestly	Better sample use	4–10 epochs	Still on-policy

5. Algorithm / Working Process

Collect a fixed rollout with current policy and value function. Save actions, rewards, done flags, values, and old log-probabilities. Compute GAE and return targets. For several epochs, shuffle into minibatches; recompute log-probabilities, form ratios, minimize clipped actor loss plus value loss minus entropy bonus, and monitor KL/clip fraction. Discard the rollout and collect fresh data.

6. Mathematical Foundation

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} = \exp(\log \pi_\theta - \log \pi_{\text{old}}).$$

Clipped surrogate to maximize:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The minimum is pessimistic: for positive advantages it caps large increases; for negative advantages it caps large decreases. A typical minimized loss is

$$\mathcal{L} = -L^{\text{CLIP}} + c_v L_V - c_e H.$$

Clipping is a heuristic, not a strict KL guarantee.

7. Practical Implementation

```
import torch

def ppo_policy_loss(new_log_prob, old_log_prob, advantage, clip_eps=0.2):
```

```

ratio = torch.exp(new_log_prob - old_log_prob)
unclipped = ratio * advantage
clipped = torch.clamp(ratio, 1 - clip_eps, 1 + clip_eps) * advantage
loss = -torch.min(unclipped, clipped).mean()
approx_kl = (old_log_prob - new_log_prob).mean()
clip_fraction = ((ratio - 1).abs() > clip_eps).float().mean()
return loss, approx_kl, clip_fraction

old_lp = torch.tensor([-0.7, -0.4])
new_lp = torch.tensor([-0.5, -0.8], requires_grad=True)
adv = torch.tensor([1.0, -0.5])
loss, kl, clip_frac = ppo_policy_loss(new_lp, old_lp, adv)
loss.backward()
print(float(loss), float(kl), float(clip_frac))

```

8. Code Explanation

Log-probability subtraction avoids direct division of tiny probabilities. Old log-probabilities must be detached snapshots. The minimum implements the pessimistic surrogate. Approximate KL and clip fraction diagnose update size; neither is the optimization objective in basic clipped PPO.

9. Training / Evaluation

Normalize advantages, tune rollout length and minibatch size, clip gradients, and consider early stopping an epoch when KL exceeds a target. Track return, success, entropy, policy/value losses, explained variance, KL, clip fraction, ratio statistics, and steps per second. Evaluate checkpoints without reward/observation-statistic leakage.

10. Complexity and Cost

For rollout length T , K epochs, and batch size B , policy training processes roughly KT samples. Memory is $O(T)$ plus model/optimizer state. PPO is usually more sample-efficient than single-update vanilla policy gradient but less so than replay-based off-policy methods. GPUs help large networks; vectorized CPU environments often feed smaller policies.

11. Common Use Cases

Robotics/control, games, navigation, simulation optimization, multi-agent baselines, LLM RLHF, and environments with either discrete or continuous actions.

12. Common Mistakes

- Recomputing "old" log-probabilities after the policy changed
- Reversing the probability ratio
- Using action probability rather than density/log-probability for continuous actions
- Reusing rollout for too many epochs
- Carrying GAE across terminals
- Ignoring value and entropy loss scales
- Assuming clipping guarantees a hard trust region

13. Edge Cases / Limitations

PPO is on-policy and interaction-expensive. Hyperparameters can be brittle across tasks. Clipping can prematurely zero useful gradients or still permit large global KL. Reward hacking remains possible. Large action/token spaces make full model training costly.

14. Variations

PPO-Clip uses the surrogate above; PPO-Penalty adapts a KL penalty; recurrent PPO handles partial observability; multi-agent PPO uses centralized or independent critics; RLHF PPO adds a reference-policy KL penalty and reward model.

15. Related Topics

Actor-critic supplies PPO's actor and critic. **GAE** estimates advantages. **TRPO** motivates trust-region updates with a constrained objective. **RLHF** commonly uses PPO to optimize a language model against learned reward.

16. Interview Questions

1. **What problem does PPO address?** Unstable large policy-gradient updates.
2. **What is the probability ratio?** New policy likelihood divided by data-collecting old policy likelihood.
3. **Why use old log-probabilities?** They fix the behavior distribution that generated data.
4. **Explain the clipped minimum.** It chooses a pessimistic objective that removes reward for excessive favorable changes.
5. **Does PPO guarantee a KL bound?** No; clipping is a practical surrogate.
6. **Why multiple epochs?** To reuse costly on-policy rollouts within limited policy drift.
7. **What is clip fraction?** Fraction of samples whose ratios lie outside the clipping interval.
8. **Why normalize advantages?** Stable optimization scale.
9. **How is PPO used for continuous actions?** With a parameterized action density, commonly a squashed/controlled Gaussian.
10. **PPO versus DQN?** On-policy direct actor-critic for either action type versus off-policy Q-learning for discrete actions.

17. Practice Tasks

- Implement PPO-Clip on CartPole.
- Plot ratio distributions and clip fraction.
- Ablate GAE lambda and number of epochs.
- Add KL early stopping.
- Debug stale/recomputed old log-probabilities.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
PPO from scratch	Solves discrete and continuous control	PyTorch, Gymnasium	CartPole/Pendulum	High placement value

Project	What it does	Tech stack	Dataset/environment	Resume value
PPO diagnostics dashboard	Tracks KL, ratios, GAE	PyTorch, Plotly	LunarLander	Strong debugging maturity
Robot control PPO	Controls simulated locomotion	PyTorch, MuJoCo/PyBullet	Locomotion benchmark	Research/robotics signal

19. Quick Revision

- **Key idea:** reuse on-policy data while clipping incentives for large probability changes.
- **Main formula:** $\min(r\hat{A}, \text{clip}(r, 1 - \epsilon, 1 + \epsilon)\hat{A})$.
- **When to use:** stable general-purpose neural policy optimization.
- **Metrics:** return, KL, clip fraction, entropy, explained variance.
- **Common trap:** not freezing old log-probabilities.
- **Interview one-liner:** "PPO takes several cautious gradient steps on each fresh rollout."

20. Final Cheat Sheet

Item	Summary
Definition	Clipped on-policy actor-critic
Input/output	Fresh rollouts → updated actor/critic
Main steps	Collect, GAE, minibatch epochs, clip, refresh
Hyperparameters	LR, clip ϵ , γ , λ , epochs, entropy/value weights
Metrics	Return, KL, clip fraction, entropy, explained variance
Pros/cons	Stable and versatile / on-policy and compute-heavy
Best uses	Discrete/continuous control and RLHF

A2C/A3C

1. Overview

Advantage Actor-Critic (A2C) and Asynchronous Advantage Actor-Critic (A3C) are parallel on-policy actor-critic algorithms. A3C introduced multiple workers that update shared parameters asynchronously; A2C synchronously batches rollouts from parallel environments for simpler, hardware-friendly training.

2. Intuition

Several players explore separate copies of a game. In A3C they independently send updates to a shared model whenever ready. In A2C they play the same number of steps, gather together, and make one averaged update.

3. Prerequisites

Actor-critic, n-step returns, advantages, parallel environments, multiprocessing, gradient descent, and on-policy learning.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
n-step return	Several rewards plus bootstrap	Faster credit propagation	5-step rollout	Bias-variance
Parallel actors	Independent environment copies	Diverse, less-correlated data	16 workers	Throughput
A3C async update	Workers update shared model	Removes strict barrier	Hogwild-style SGD	Stale gradients
A2C sync batch	Wait and aggregate workers	Stable vectorized compute	One batched update	Difference from A3C
Entropy regularization	Maintains exploration	Avoids collapse	Policy entropy bonus	Loss

5. Algorithm / Working Process

Each worker rolls out current policy for n steps or until terminal. Bootstrap the final nonterminal state value, compute returns backward, derive advantages, and calculate actor, critic, and entropy terms. A2C averages gradients across synchronous workers before one optimizer step. A3C applies each worker's gradients asynchronously to shared parameters, then refreshes local weights.

6. Mathematical Foundation

n-step return:

$$\hat{G}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n V_\phi(S_{t+n})$$

when no terminal occurs. Advantage is $\hat{A}_t = \hat{G}_t^{(n)} - V_\phi(S_t)$. Combined minimized loss:

$$\mathcal{L} = -\mathbb{E}[\log \pi_\theta(a_t | s_t) \hat{A}_t] + c_v \frac{1}{2} \mathbb{E}[(V_\phi(s_t) - \hat{G}_t)^2] - c_e \mathbb{E}[H(\pi_\theta)].$$

Synchronous gradients approximate a batch mean; asynchronous gradients may be stale.

7. Practical Implementation

```
import torch

def n_step_returns(rewards, terminated, last_value, gamma=0.99):
```

```

"""rewards/terminated shape: [time, environments]."""
returns = torch.zeros_like(rewards)
running = last_value
for t in reversed(range(rewards.shape[0])):
    running = rewards[t] + gamma * (1 - terminated[t].float()) * running
    returns[t] = running
return returns

rewards = torch.tensor([[1.0, 0.0], [0.0, 1.0], [1.0, 1.0]])
terminated = torch.tensor([[False, False], [False, True], [True, False]])
last_value = torch.tensor([0.0, 0.4])
print(n_step_returns(rewards, terminated, last_value))

```

8. Code Explanation

The tensor holds multiple environments in columns and time in rows. Backward recursion bootstraps from each worker's last value but resets wherever that environment terminates. A2C feeds these targets into a single batched actor-critic update.

9. Training / Evaluation

Track rollout throughput, return per environment, entropy, policy/value loss, explained variance, gradient norm, and worker imbalance. Seed workers differently. In A3C, watch parameter staleness and nondeterminism; in A2C, watch slow workers and synchronization barriers.

10. Complexity and Cost

With N workers and T -step rollouts, an A2C update uses NT samples. Vectorization improves throughput but stores $O(NT)$ tensors. A3C uses CPU multiprocessing and shared memory; A2C maps naturally to SIMD/GPU batches. Environment speed often decides optimal worker count.

11. Common Use Cases

Classic control, Atari, CPU-heavy simulators, navigation, distributed RL education, and baselines where PPO's multiple epochs are unnecessary.

12. Common Mistakes

- Saying A2C means "asynchronous"
- Sharing identical random seeds across workers
- Bootstrapping past terminals
- Mixing states from workers when computing returns
- Using stale local parameters indefinitely in A3C
- Assuming more workers always increase effective sample efficiency

13. Edge Cases / Limitations

Both are on-policy and discard old rollouts. A3C is nondeterministic and harder to debug. A2C can idle on slow environments. Correlated workers reduce the diversity benefit. They usually provide less rollout reuse

than PPO.

14. Variations

Recurrent A2C/A3C handles memory; ACKTR uses Kronecker-factored natural-gradient approximations; IMPALA scales actors and corrects behavior lag using V-trace; PPO adds clipped multi-epoch optimization.

15. Related Topics

Actor-critic gives the loss. **Advantage/GAE** improves policy signals. **PPO** usually adds controlled data reuse. **Distributed RL** generalizes parallel actor/learner architectures.

16. Interview Questions

1. **What does A3C stand for?** Asynchronous Advantage Actor-Critic.
2. **What does A2C stand for?** Advantage Actor-Critic, the synchronous counterpart.
3. **Main difference?** A3C applies worker updates asynchronously; A2C batches synchronized rollouts.
4. **Why parallel workers?** Higher throughput and more diverse, less-correlated on-policy samples.
5. **What target is used?** Usually an n-step bootstrapped return.
6. **Why entropy bonus?** To preserve exploration.
7. **What is gradient staleness?** A3C worker gradients were computed using older parameters.
8. **Why might A2C be faster on GPUs?** Synchronous data forms efficient batches.
9. **Are these off-policy?** Standard A2C/A3C are on-policy.
10. **A2C versus PPO?** PPO commonly reuses each rollout for clipped minibatch epochs; A2C usually makes one update.

17. Practice Tasks

- Implement vectorized n-step returns.
- Train A2C with 1, 4, and 16 environments.
- Check worker-specific terminal masks.
- Compare A2C and PPO at equal environment steps.
- Profile environment versus learner bottlenecks.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Vectorized A2C	Trains parallel classic-control agents	PyTorch, Gymnasium	CartPole/LunarLander	Parallel RL implementation
Async/sync benchmark	Measures A3C versus A2C	PyTorch multiprocessing	Atari-lite	Systems + ML insight
Recurrent A2C	Solves partial-observation navigation	PyTorch, MiniGrid	MiniGrid	Memory/control competence

19. Quick Revision

- **Key idea:** parallel actor-critics learn from n-step advantages.
- **Main formula:** $G_t^{(n)} - V(s_t)$.
- **When to use:** simple parallel on-policy training.
- **Metrics:** return, throughput, entropy, explained variance.
- **Common trap:** reversing which method is asynchronous.
- **Interview one-liner:** "A3C updates shared weights asynchronously; A2C synchronizes workers into one batch."

20. Final Cheat Sheet

Item	Summary
Definition	Parallel advantage actor-critic methods
Input/output	Multi-worker rollouts → shared model update
Main steps	Roll out, n-step target, advantage, actor/critic update
Hyperparameters	Worker count, n-step, LR, entropy/value weights
Metrics	Return, steps/s, entropy, value quality
Pros/cons	Simple parallelism / on-policy and sync/staleness issues
Best uses	Vectorized control baselines

Model-Based RL

1. Overview

Model-based reinforcement learning uses a model of environment dynamics and rewards to plan, generate imagined experience, or improve a policy. The model may be given, learned from data, deterministic, probabilistic, pixel-level, or latent. It can greatly improve interaction efficiency but introduces model bias.

2. Intuition

Before moving a chess piece, a player imagines possible replies. A model-based agent similarly predicts "if I do this, what happens next?" and selects actions using simulated futures rather than learning only from real trial and error.

3. Prerequisites

MDPs, supervised learning, probability models, planning/search, value functions, uncertainty, neural networks, and optimization.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Dynamics model	Predicts s' from s, a	Enables imagination	Robot next pose	Deterministic/probabilistic
Reward model	Predicts immediate reward	Scores plans	Energy cost	Learned versus known
Planning	Optimizes actions through model	Uses lookahead	MPC shooting	Planning horizon
Model bias	Systematic prediction error	Compounds over rollout	Unrealistic imagined states	Exploitation
Uncertainty	Confidence in predictions	Prevents risky extrapolation	Ensemble disagreement	Epistemic/aleatoric

5. Algorithm / Working Process

Collect real transitions, fit dynamics/reward models, and validate them. At decision time, generate candidate action sequences, roll the model forward, score predicted returns plus terminal value, execute only the first action (model-predictive control), observe reality, add the transition, and replan. Alternatively, create synthetic transitions to train a model-free learner as in Dyna.

6. Mathematical Foundation

Learn parameters by minimizing, for deterministic dynamics,

$$\mathcal{L}_{\text{dyn}}(\phi) = \mathbb{E} [\|f_{\phi}(s_t, a_t) - s_{t+1}\|^2] .$$

A probabilistic model minimizes negative log-likelihood:

$$\mathcal{L}_{\text{NLL}} = -\mathbb{E}[\log p_{\phi}(s_{t+1}, r_{t+1} \mid s_t, a_t)] .$$

Planning chooses

$$a_{t:t+H-1}^* = \arg \max \mathbb{E}_{\hat{p}_{\phi}} \left[\sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k} + \gamma^H V(s_{t+H}) \right] .$$

Longer horizon uses more foresight but compounds model error.

7. Practical Implementation

```
import numpy as np

def random_shooting(model, state, action_low, action_high,
                    candidates=1_000, horizon=20, gamma=0.99):
    action_sequences = np.random.uniform(
        action_low, action_high, size=(candidates, horizon, len(action_low))
    )
```

```

states = np.repeat(state[None, :], candidates, axis=0)
scores = np.zeros(candidates)
discount = 1.0
for t in range(horizon):
    states, rewards = model(states, action_sequences[:, t])
    scores += discount * rewards
    discount *= gamma
best = np.argmax(scores)
return action_sequences[best, 0] # MPC executes only first action

```

8. Code Explanation

Random shooting samples candidate action sequences within bounds, rolls all candidates through a vectorized learned model, and chooses the first action from the highest-scoring sequence. Replanning after every real step limits the effect of later model errors.

9. Training / Evaluation

Split by trajectories or scenarios, not randomly adjacent transitions only. Measure one-step and multi-step prediction, calibration, ensemble disagreement, planning return, and real-world return. Test model exploitation by inspecting high-scoring imagined rollouts. Continually refresh the model with newly visited states.

10. Complexity and Cost

Planning cost is roughly $O(CH)$ model evaluations for C candidates and horizon H , often vectorized on GPU. Learned models add training and storage but can reduce expensive real interactions. Pixel world models require substantial compute; compact latent models reduce rollout cost.

11. Common Use Cases

Robotics, autonomous control, industrial processes, games, data-efficient experimentation, planning in simulators, and learned world models.

12. Common Mistakes

- Optimizing one-step prediction while ignoring multi-step control error
- Planning far beyond reliable model support
- Letting the planner exploit model inaccuracies
- Randomly splitting adjacent transitions and leaking trajectories
- Ignoring uncertainty outside the dataset
- Comparing imagined return with real return as if equivalent

13. Edge Cases / Limitations

Chaotic, stochastic, and partially observed dynamics are hard to model. Small errors compound. A planner actively seeks optimistic model mistakes. Discontinuities and contact dynamics challenge differentiable models. Planning latency can violate real-time limits.

14. Variations

Dyna mixes real and imagined updates; MPC replans online; PETS uses probabilistic ensembles; MBPO uses short model rollouts; Dreamer learns latent world models; AlphaZero uses known dynamics with search; MuZero learns search-relevant latent dynamics without reconstructing observations.

15. Related Topics

MDPs define the modeled dynamics. **Offline RL** can use models but must avoid extrapolation. **MuZero** combines learned latent models and MCTS. **Model-free RL** skips explicit transition prediction.

16. Interview Questions

1. **Model-based versus model-free?** Model-based methods predict dynamics/reward for planning or synthetic data; model-free methods directly learn policy/value.
2. **Why more sample-efficient?** One real transition can train a reusable model and support many imagined rollouts.
3. **What is model bias?** Control error caused by inaccurate learned dynamics.
4. **Why MPC?** Replanning after each real observation corrects accumulated prediction error.
5. **Deterministic versus probabilistic model?** Point prediction versus distribution over outcomes.
6. **What do ensembles provide?** Approximate epistemic uncertainty and robustness.
7. **Why short model rollouts?** Error compounds with imagined horizon.
8. **What is Dyna?** A framework mixing real experience with model-generated updates.
9. **One-step loss sufficient?** No; good one-step prediction may still produce poor long-horizon decisions.
10. **What is latent model learning?** Predicting task-relevant hidden representations rather than raw observations.

17. Practice Tasks

- Learn cart dynamics from transitions.
- Implement random-shooting MPC.
- Plot one-step versus multi-step error.
- Demonstrate planner exploitation of a biased model.
- Add an ensemble uncertainty penalty.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Learned MPC controller	Controls system with fitted dynamics	PyTorch, Gymnasium	Pendulum transitions	Strong controls/ML crossover
Dyna-Q navigator	Mixes real and imagined updates	NumPy	GridWorld	Clear model-based fundamentals
Latent world model	Predicts compact future features	PyTorch, torchvision	CarRacing videos	Research internship value

19. Quick Revision

- **Key idea:** learn/use dynamics to imagine consequences before acting.
- **Main formula:** $\text{plan } \arg \max \mathbb{E}_{\hat{p}}[\sum \gamma^k r]$.
- **When to use:** real interaction is expensive and dynamics are learnable.
- **Metrics:** multi-step error, calibration, real return, planning time.
- **Common trap:** planner exploitation of model error.
- **Interview one-liner:** "Model-based RL spends computation on imagined futures to save real-world samples."

20. Final Cheat Sheet

Item	Summary
Definition	RL using explicit learned/known environment model
Input/output	State/action $\rightarrow$ predicted future; plan $\rightarrow$ action
Main steps	Collect, fit, plan/imagine, act, refresh
Hyperparameters	Model LR, ensemble size, horizon, candidates
Metrics	Multi-step error, uncertainty, real return
Pros/cons	Sample-efficient/plannable / model bias and compute
Best uses	Expensive interaction and predictable dynamics

Multi-Agent RL

1. Overview

Multi-agent reinforcement learning (MARL) studies environments containing multiple learning or strategic agents. Their rewards may be cooperative, competitive, or mixed. Each agent's learning changes the effective environment experienced by others, creating non-stationarity and difficult credit-assignment and coordination problems.

2. Intuition

In football, one player's best move depends on teammates and opponents. Training each player as if everyone else were fixed fails because all policies evolve. Good MARL methods learn coordination, opponent responses, and who contributed to a team outcome.

3. Prerequisites

Single-agent MDPs, game theory basics, joint probability, actor-critic and Q-learning, partial observability, centralized versus decentralized computation, and neural networks.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Stochastic game	Multi-agent MDP generalization	Formal MARL model	Team game	Joint transition/reward
Joint action	Tuple $\mathbf{a} = (a_1, \dots, a_n)$	Determines transition	All robot commands	Exponential size
CTDE	Centralized training, decentralized execution	Uses global data only while training	Central critic, local actors	Key paradigm
Non-stationarity	Other policies change	Breaks stationary-MDP view	Moving opponent	Replay issues
Credit assignment	Attribute team reward to agents	Enables cooperation	Which robot helped?	Counterfactual baseline
Communication	Agents exchange learned messages	Improves coordination	Robot broadcasts intent	Bandwidth/deployment

5. Algorithm / Working Process

Define per-agent observations, actions, rewards, and information available at deployment. Run all policies to obtain a joint action, step the shared environment, and store joint transitions. During centralized training, a critic may access global state and all actions. Update decentralized actors from their local observations. At inference, remove privileged centralized inputs and execute each policy independently or with allowed communication.

6. Mathematical Foundation

A Markov game can be written

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}, P, \{R_i\}, \{\mathcal{O}_i\}, \gamma \rangle.$$

Agent i seeks

$$J_i(\pi_1, \dots, \pi_n) = \mathbb{E} \left[\sum_t \gamma^t R_{i,t+1} \right].$$

A centralized Q critic may be

$$Q_i(s, a_1, \dots, a_n),$$

while actor i uses only $\mathcal{O}_i$. In cooperative value decomposition, Q_{tot} is constructed from individual utilities. QMIX imposes $\partial Q_{\text{tot}} / \partial Q_i \geq 0$, allowing decentralized argmax while modeling nonlinear mixing.

7. Practical Implementation

```

import torch
from torch import nn

class CentralCritic(nn.Module):
    """Centralized critic; decentralized actors remain separate."""
    def __init__(self, global_state_dim, joint_action_dim, n_agents):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(global_state_dim + joint_action_dim, 128), nn.ReLU(),
            nn.Linear(128, n_agents)
        )

    def forward(self, global_state, joint_one_hot_action):
        features = torch.cat([global_state, joint_one_hot_action], dim=-1)
        return self.net(features) # one value estimate per agent

critic = CentralCritic(global_state_dim=12, joint_action_dim=8, n_agents=2)
values = critic(torch.randn(32, 12), torch.randn(32, 8))
print(values.shape) # [batch, agents]

```

8. Code Explanation

The centralized critic conditions on information that individual actors may not observe: global state and all actions. It outputs per-agent evaluations. This is only a component; a full CTDE system also defines local actors, joint rollout storage, targets, and actor updates.

9. Training / Evaluation

Evaluate team return, per-agent return, win rate, fairness, coordination success, communication cost, robustness to unseen teammates/opponents, and exploitability where applicable. Split evaluation by opponent policies and team compositions. Run many seeds because equilibrium selection and co-adaptation are unstable.

10. Complexity and Cost

Naive joint action enumeration is $O(\prod_i |\mathcal{A}_i|)$. Central critics grow with agent count and joint information. Simulation and communication overhead scale with agents. Parameter sharing reduces memory for homogeneous agents; population training multiplies compute but improves robustness.

11. Common Use Cases

Robot fleets, autonomous driving interactions, traffic control, multiplayer games, auctions/markets, network routing, warehouse coordination, drone swarms, and distributed resource allocation.

12. Common Mistakes

- Treating other learning agents as stationary dynamics
- Giving decentralized actors privileged training-only state at deployment
- Replaying very old joint data without correcting policy drift

- Reporting only team reward and hiding one agent's failure
- Assuming parameter sharing makes agents interchangeable without IDs/roles
- Evaluating only against training teammates/opponents

13. Edge Cases / Limitations

Joint spaces explode; equilibrium may not be unique; self-play can cycle. Agents may develop brittle conventions that fail with new partners. Communication may be unavailable or delayed. Competitive training can produce arms races and exploit simulator artifacts.

14. Variations

Independent Q-learning treats agents separately; MADDPG uses centralized Q critics with decentralized continuous actors; MAPPO applies PPO with centralized critics; VDN sums individual utilities; QMIX uses monotonic mixing; COMA uses counterfactual advantages; self-play and population-based training target competitive robustness.

15. Related Topics

Game theory supplies Nash equilibrium and best responses. **POMDPs** arise because an agent sees only local information. **Actor-critic** enables CTDE. **Offline MARL** adds fixed-data coverage and multi-policy confounding.

16. Interview Questions

1. **Why is MARL non-stationary?** Each agent's policy changes transition/reward distributions seen by others.
2. **What is CTDE?** Use centralized/global information for training critics, but local information for deployed actors.
3. **What is a joint action?** The tuple of all agents' simultaneous actions.
4. **Why does action space explode?** Joint choices grow as the product of individual action counts.
5. **Cooperative versus competitive?** Shared/aligned rewards versus opposing objectives; mixed settings combine both.
6. **What is credit assignment?** Determining each agent's contribution to shared outcome.
7. **Why can replay be harmful?** Stored transitions were generated by older combinations of policies.
8. **What does QMIX guarantee structurally?** Monotonic mixing makes decentralized individual argmax consistent with global argmax.
9. **Why test unseen teammates?** To distinguish general coordination from co-adapted conventions.
10. **What is self-play?** Training agents against current or historical versions of themselves.

17. Practice Tasks

- Build a two-agent cooperative gridworld.
- Compare independent Q-learning and centralized critic.
- Measure generalization to unseen partner policies.
- Diagnose a privileged-information leak at execution.
- Add learned communication with a bandwidth limit.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Warehouse fleet	Coordinates multiple pickers	PyTorch, PettingZoo	Multi-agent grid	Strong applied coordination
MAPPO team game	Learns centralized-critic policies	PyTorch, PettingZoo	MPE/SMAC-like task	Research-oriented MARL
Self-play tournament	Trains and evaluates populations	PyTorch, Elo tooling	Connect Four	Game theory and evaluation

19. Quick Revision

- **Key idea:** multiple adapting policies jointly determine outcomes.
- **Main formula:** $J_i(\pi_1, \dots, \pi_n)$.
- **When to use:** strategic, cooperative, or competitive multi-actor systems.
- **Metrics:** team/per-agent return, win rate, robustness, exploitability.
- **Common trap:** centralized information leaking into decentralized execution.
- **Interview one-liner:** "MARL turns a stationary control problem into a moving game among learners."

20. Final Cheat Sheet

Item	Summary
Definition	RL with interacting learning/strategic agents
Input/output	Per-agent observations → joint actions
Main steps	Joint rollout, centralized/independent evaluation, policy updates
Hyperparameters	Agent count, communication, population size, PPO/Q settings
Metrics	Team return, win rate, coordination, generalization
Pros/cons	Learns coordination/competition / non-stationary and combinatorial
Best uses	Fleets, games, traffic, markets

Offline RL

1. Overview

Offline or batch reinforcement learning learns a policy from a fixed dataset of logged transitions without additional environment interaction during training. It is valuable when exploration is unsafe, expensive, or impossible, but must handle distribution shift: learned policies may choose actions poorly represented in the data.

2. Intuition

Learning to treat patients only from historical records is not like a simulator where the algorithm can try arbitrary treatments. A naive policy may recommend a rarely recorded action because an unreliable Q-

function predicts it is excellent. Offline RL must prefer improvements supported by evidence.

3. Prerequisites

Off-policy RL, Q-learning, behavior policies, importance sampling, supervised learning, distribution shift, uncertainty, and causal/confounding awareness.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Fixed dataset $\mathcal{D}$	No active collection	Defines all support	Logged trajectories	Offline versus off-policy
Behavior policy μ	Generated data	Determines coverage	Clinician decisions	Unknown behavior
Support mismatch	Learned policy chooses unseen actions	Causes extrapolation error	Rare treatment	Central difficulty
Conservative learning	Penalizes OOD actions/values	Avoids optimism	CQL penalty	Safety
Behavior regularization	Keeps π near data	Reduces shift	KL/BC constraint	Trade-off
OPE	Estimates policy without deployment	Needed for selection	Importance sampling	High variance

5. Algorithm / Working Process

Audit dataset provenance, coverage, terminals, confounding, and reward definition. Split by trajectory/entity/time. Train behavior cloning as a baseline. Train a conservative value or policy method using only fixed minibatches. Select checkpoints with offline policy evaluation and simulator tests if credible. Deploy gradually with safeguards and monitor shift; training-time online exploration is absent.

6. Mathematical Foundation

Naive Bellman regression uses

$$\mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(Q(s,a) - r - \gamma \max_{a'} Q_{\text{target}}(s',a') \right)^2 \right].$$

The max may choose unsupported a' . Conservative Q-Learning adds, schematically,

$$\alpha \left(\mathbb{E}_{s \sim \mathcal{D}, a \sim \pi} [Q(s,a)] - \mathbb{E}_{(s,a) \sim \mathcal{D}} [Q(s,a)] \right),$$

often implemented with log-sum-exp over discrete actions. Behavior-regularized methods constrain $D(\pi(\cdot | s), \mu(\cdot | s))$. Importance-sampling OPE weights trajectories by products of $\pi(a_t | s_t) / \mu(a_t | s_t)$, which can have extreme variance.

7. Practical Implementation

```
import torch
from torch.nn import functional as F

def discrete_cql_loss(q_net, target_net, batch, gamma=0.99, alpha=1.0):
    states, actions, rewards, next_states, terminated = batch
    all_q = q_net(states)
    data_q = all_q.gather(1, actions[:, None]).squeeze(1)
    with torch.no_grad():
        next_q = target_net(next_states).max(1).values
        target = rewards + gamma * (1 - terminated.float()) * next_q
    bellman = F.smooth_l1_loss(data_q, target)
    conservative = (torch.logsumexp(all_q, dim=1) - data_q).mean()
    return bellman + alpha * conservative
```

8. Code Explanation

Bellman loss learns from logged actions. The conservative term raises a log-sum-exp over all actions and subtracts logged-action values; minimizing it pushes unsupported action values down relative to dataset actions. This compact discrete form illustrates CQL's idea but a production method needs target updates, tuning, and careful evaluation.

9. Training / Evaluation

Split complete trajectories and users/patients to prevent leakage. Report dataset-return baselines, behavior cloning, Fitted Q Evaluation, importance-sampling estimates with effective sample size, simulator return when valid, and uncertainty. Hyperparameters include conservative weight, behavior constraint, discount, target rate, and model capacity. Overfitting appears as improving training Bellman loss with deteriorating OPE.

10. Complexity and Cost

Training resembles off-policy deep RL but avoids environment cost. Dataset storage and repeated epochs dominate. Discrete CQL log-sum-exp is $O(|A|)$; continuous versions sample actions. OPE can be statistically expensive even when computation is modest. CPU/GPU need depends on observation/model size.

11. Common Use Cases

Healthcare, autonomous-driving logs, industrial control, recommender logs, robotics demonstrations, operations data, and any safety-critical domain where online random exploration is unacceptable.

12. Common Mistakes

- Saying offline RL can explore during training
- Randomly splitting transitions from the same trajectories/users
- Using max-Q naively on unseen actions
- Choosing checkpoints solely by Bellman loss
- Ignoring behavior cloning baseline

- Claiming causal treatment quality from confounded observational data
- Evaluating on the same logged actions used for training

13. Edge Cases / Limitations

No algorithm can identify outcomes of completely unsupported actions without assumptions. Hidden confounding can invalidate policy conclusions. OPE may have huge variance. Narrow datasets force conservative policies. Non-stationary deployment can leave training support.

14. Variations

Behavior cloning is pure imitation; BCQ restricts candidate actions; BEAR constrains distribution distance; CQL learns conservative values; IQL avoids explicit evaluation of unseen actions using expectile regression; Decision Transformer frames trajectories as conditional sequence modeling; model-based offline RL penalizes uncertainty.

15. Related Topics

Off-policy RL may still interact online; offline RL may not. **Imitation learning** copies behavior rather than explicitly improving return. **Causal inference** addresses confounding. **RLHF** is often partly offline when learning rewards/preferences, although PPO rollout generation is online relative to the model.

16. Interview Questions

1. **Offline versus off-policy RL?** Offline uses a fixed dataset only; off-policy algorithms can still collect new data.
2. **Central challenge?** Distribution shift and value extrapolation on unsupported actions.
3. **Why does max-Q hurt?** It selects actions whose optimistic errors are not corrected by data.
4. **What is behavior regularization?** Constraining learned policy near logged behavior.
5. **What does CQL do?** Penalizes high values on actions not supported relative to dataset actions.
6. **Why use behavior cloning baseline?** It is stable and reveals whether RL adds real value.
7. **What is OPE?** Estimating a new policy's performance without executing it.
8. **Why high-variance importance sampling?** Trajectory likelihood ratios multiply over time.
9. **Can offline RL discover an unseen optimal action?** Not reliably without structural/model assumptions.
10. **Why split by trajectory or user?** Adjacent/shared-entity transitions otherwise leak information.

17. Practice Tasks

- Train behavior cloning from logged grid trajectories.
- Show naive DQN overestimation on missing actions.
- Add the discrete CQL penalty.
- Compute importance weights and effective sample size.
- Audit an offline dataset for support and leakage.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Offline CartPole	Compares BC, DQN, CQL	PyTorch, Gymnasium	Fixed policy rollouts	Clear offline RL benchmark
Logged recommender	Learns conservative ranking policy	PyTorch, Pandas	KuaiRec/MovieLens simulator	Industry relevance
Clinical policy audit	Evaluates support/confounding risks	Pandas, PyTorch	MIMIC-style deidentified data	Responsible AI value

19. Quick Revision

- **Key idea:** improve from fixed logs without choosing unsupported actions.
- **Main formula:** Bellman loss plus conservative/support regularizer.
- **When to use:** interaction is unsafe, costly, or unavailable.
- **Metrics:** OPE, simulator return, support, effective sample size.
- **Common trap:** selecting high-Q out-of-distribution actions.
- **Interview one-liner:** “Offline RL’s problem is not replay—it is proving improvement without new corrective experience.”

20. Final Cheat Sheet

Item	Summary
Definition	Policy learning from a fixed transition dataset
Input/output	Logged trajectories → deployable policy
Main steps	Audit, split, BC baseline, conservative train, OPE
Hyperparameters	Conservatism, behavior constraint, γ , target rate
Metrics	OPE, support, ESS, downstream safety
Pros/cons	No risky exploration / identifiability and shift
Best uses	Logged safety-critical domains

RLHF

1. Overview

Reinforcement Learning from Human Feedback (RLHF) aligns a model’s behavior with human preferences. A common LLM pipeline starts from a pretrained model, performs supervised fine-tuning (SFT), learns a reward model from ranked responses, and optimizes the policy—often with PPO—while constraining drift from a reference model. RLHF is a family of pipelines, not one algorithm.

2. Intuition

It is difficult to write a formula for “helpful, clear, and safe,” but people can often choose the better of two answers. A reward model learns these comparisons; the language model then learns to produce responses that receive higher predicted preference while staying close to its capable pretrained behavior.

3. Prerequisites

Transformers and autoregressive language modeling, token probabilities, cross-entropy/SFT, preference data, Bradley–Terry models, policy gradients/PPO, KL divergence, reward modeling, distributed training, and AI safety evaluation.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
SFT policy	Model trained on demonstrations	Strong starting behavior	Instruction-response pairs	Why before RL
Preference data	Ranked response pairs	Converts judgment to supervision	Chosen vs rejected	Label quality
Reward model	Scores prompt-response sequences	Proxy for human preference	Scalar at final token	Pairwise loss
Reference policy	Frozen baseline	Limits capability/language drift	SFT checkpoint	KL penalty
Policy optimization	Improves expected learned reward	Produces aligned policy	PPO updates	Reward hacking
Human evaluation	Direct assessment	Validates proxy	Blind pairwise win rate	Gold standard limits

5. Algorithm / Working Process

1. Pretrain an autoregressive model on broad text.
2. Supervised-fine-tune it on high-quality demonstrations.
3. For prompts, sample multiple responses and collect human rankings.
4. Train a reward model to score chosen responses above rejected ones.
5. Generate fresh responses from the current policy.
6. Compute reward-model score plus a penalty for divergence from the frozen reference.
7. Estimate token-level advantages with a critic and update with PPO or another optimizer.
8. Repeatedly evaluate helpfulness, safety, factuality, diversity, and reward-model exploitation.

At inference, only the final policy is normally required; reward and critic models are training infrastructure.

6. Mathematical Foundation

For chosen response y_w and rejected response y_l , a Bradley–Terry reward-model loss is

$$\mathcal{L}_{\text{RM}} = -\mathbb{E} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))].$$

A KL-regularized policy objective is

$$\max_{\theta} \mathbb{E}_{x, y \sim \pi_{\theta}} \left[r_{\phi}(x, y) - \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} \right].$$

For autoregressive models,

$$\log \pi_{\theta}(y | x) = \sum_t \log \pi_{\theta}(y_t | x, y_{<t}),$$

so KL-related rewards can be assigned tokenwise. PPO then optimizes a clipped policy ratio using advantages. Direct Preference Optimization (DPO) instead turns preference data into a supervised classification-style objective relative to a reference policy, avoiding online RL rollouts and a separate deployed reward model during optimization.

7. Practical Implementation

```
import torch
from torch.nn import functional as F

def reward_model_pairwise_loss(chosen_scores, rejected_scores):
    """Bradley-Terry negative log-likelihood."""
    return -F.logsigmoid(chosen_scores - rejected_scores).mean()

def kl_shaped_sequence_reward(reward_score, policy_log_probs,
                              reference_log_probs, beta=0.02):
    """Token rewards: KL penalty each token, preference reward at end."""
    token_rewards = -beta * (policy_log_probs - reference_log_probs)
    token_rewards = token_rewards.clone()
    token_rewards[:, -1] += reward_score
    return token_rewards

chosen = torch.tensor([2.0, 0.2], requires_grad=True)
rejected = torch.tensor([0.5, -0.1], requires_grad=True)
loss = reward_model_pairwise_loss(chosen, rejected)
loss.backward()
print(float(loss))
```

8. Code Explanation

The pairwise loss depends only on score difference, encouraging chosen responses to rank higher. The shaping function subtracts a per-token log-ratio penalty relative to a frozen reference and adds the preference score at sequence end. A full system also handles masks, variable lengths, value estimation, distributed generation, and PPO minibatches.

9. Training / Evaluation

Split by prompt source, annotator, and time where relevant to prevent near-duplicate leakage. Reward-model metrics include pairwise accuracy, calibration, subgroup analysis, and robustness to length/style shortcuts. Policy metrics include blind human win rate, task benchmarks, safety/red-team suites, factuality, diversity, KL to reference, length, reward score, and disagreement between reward models. High reward-model score alone is not valid evaluation because the policy optimizes that proxy.

10. Complexity and Cost

RLHF can hold four large components: policy, reference, reward model, and critic, plus optimizer states. Autoregressive rollout generation is sequential and expensive; PPO adds multiple training passes. Memory is reduced with parameter-efficient tuning, sharding, mixed precision, and shared backbones. Human comparison collection is a significant cost and quality bottleneck.

11. Common Use Cases

Helpful and safe assistants, instruction following, summarization style, dialogue quality, code-generation preferences, tool-use behavior, refusal calibration, and domain-specific response policies.

12. Common Mistakes

- Calling ordinary supervised instruction tuning RLHF
- Treating reward-model score as ground-truth quality
- Training/evaluating on near-duplicate prompts
- Omitting or miscomputing reference KL
- Ignoring annotator disagreement and demographic bias
- Allowing length/style shortcuts to dominate rewards
- Evaluating only average helpfulness and hiding safety regressions

13. Edge Cases / Limitations

Human preferences are inconsistent, culturally dependent, and expensive. Reward models generalize imperfectly and can be gamed. KL constraints limit but do not prevent collapse. Pairwise rankings may not capture factual truth or rare catastrophic risk. Strong policies can move outside reward-model training distribution.

14. Variations

PPO-based RLHF uses online policy rollouts and a reward model. Rejection sampling selects high-scoring candidates. DPO and related direct preference methods optimize preference likelihood without a PPO loop. RLAIF uses AI-generated feedback. Constitutional AI uses explicit principles and critique/revision. Process reward models score reasoning steps rather than only final responses.

15. Related Topics

PPO is a common optimizer. **Offline RL** shares fixed logged preference/data concerns. **Reward modeling** learns the proxy. **DPO** is a direct alternative. **LLM fine-tuning**, red teaming, calibration, and scalable oversight complete the alignment pipeline.

16. Interview Questions

1. **What is RLHF?** A pipeline that uses human preferences to train a reward/objective and improve a policy.
2. **Typical stages?** Pretraining, SFT, preference collection, reward modeling, constrained policy optimization, evaluation.
3. **Why SFT first?** It provides capable instruction-following behavior and reduces unsafe/random RL exploration.
4. **Reward-model loss?** Pairwise logistic loss on chosen versus rejected response scores.
5. **Why a reference policy?** It anchors language quality and limits policy drift through KL regularization.
6. **Why not trust RM score alone?** The policy can exploit reward-model errors and shortcuts.
7. **Where is the reward assigned?** Preference score is often sequence-level; KL shaping can be token-level and advantages distribute credit.
8. **What is DPO?** A direct preference objective comparing policy/reference log-ratios for chosen and rejected responses.
9. **RLHF versus supervised fine-tuning?** SFT imitates demonstrations; RLHF optimizes preference-derived sequence reward.
10. **Major failure modes?** Reward hacking, mode collapse, verbosity bias, sycophancy, annotator bias, and safety regressions.

17. Practice Tasks

- Implement pairwise reward-model loss.
- Train a tiny text scorer on chosen/rejected samples.
- Inspect reward correlation with response length.
- Compute tokenwise policy/reference log-ratios.
- Compare rejection sampling and a direct preference objective on a toy model.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Preference reward model	Ranks helpful responses	PyTorch, Transformers	Anthropic HH-style/public preferences	Direct alignment experience
Tiny DPO assistant	Tunes a small LM on preferences	Hugging Face TRL/Transformers	UltraFeedback subset	Strong LLM post-training signal
Reward-hacking audit	Finds length/style shortcuts	Pandas, PyTorch, SHAP/plots	Public preference pairs	Responsible AI/research value

19. Quick Revision

- **Key idea:** learn human preference signal, then optimize policy while limiting drift.
- **Main formula:** pairwise $-\log \sigma(r_w - r_l)$; objective $r - \beta \text{KL}$.
- **When to use:** qualities easier to compare than specify exactly.
- **Metrics:** human win rate, safety, factuality, RM accuracy, KL, diversity.
- **Common trap:** equating learned reward with true human value.

- **Interview one-liner:** “RLHF converts comparative human judgment into a constrained policy-improvement signal.”

20. Final Cheat Sheet

Item	Summary
Definition	Preference-based model post-training pipeline
Input/output	Prompts, rankings, models → aligned policy
Main steps	SFT, rank data, RM, KL-constrained optimize, human eval
Hyperparameters	KL β , PPO clip/LR, generation temperature, RM LR
Metrics	Human win rate, safety, RM accuracy, KL, diversity
Pros/cons	Optimizes nuanced preferences / costly and proxy-exploitable
Best uses	Assistant behavior and LLM alignment

MuZero-Style Methods

1. Overview

MuZero-style methods combine learned latent dynamics, value/policy/reward prediction, and Monte Carlo Tree Search (MCTS). Unlike classical model-based methods that predict future observations, MuZero learns only the aspects of a hidden state needed for planning: rewards, policies, and values. It achieved strong results in board games and Atari-like domains without being given environment transition rules.

2. Intuition

A skilled chess player does not need to imagine a photorealistic future board image; they need a compact mental representation that supports legal-looking consequences and position evaluation. MuZero learns such a planning representation from experience and uses search to improve decisions.

3. Prerequisites

Value and policy networks, model-based RL, MCTS/UCT, residual neural networks, categorical value representations, replay buffers, bootstrapping, self-play, and distributed systems for large implementations.

4. Core Concepts

Subtopic	What it means	Why it matters	Simple example	Common interview angle
Representation h	Observation history → latent state	Search starts in compact space	Encoded game position	No reconstruction requirement
Dynamics g	Latent state + action → next	Enables imagined tree	Hidden transition	Learned model

Subtopic	What it means	Why it matters	Simple example	Common interview angle
latent/reward				
Prediction f	Latent $\rightarrow$ policy and value	Guides/evaluates search	Move priors and position value	Network heads
MCTS	Select, expand, evaluate, back up	Policy improvement by planning	Repeated move simulations	PUCT
Search policy target	Visit-count distribution	Trains better policy than raw network	Root visits	Expert iteration
Reanalysis	Rerun search/value on replay	Improves old targets	Updated MCTS labels	Compute/data trade-off

5. Algorithm / Working Process

Encode recent observations with $s^0 = h_\theta(o_{\leq t})$. Run MCTS from s^0 : select actions using value plus exploration prior, expand a leaf using g_θ , evaluate its policy/value using f_θ , and back up predicted rewards and value. Sample or choose the real action from root visit counts. Store trajectory. During training, unroll dynamics for K recorded actions and predict reward, value, and policy targets derived from environment outcomes and search. Repeat with self-play/data generation.

At inference, representation, dynamics, prediction, and search are used; training-only losses and replay targets are absent.

6. Mathematical Foundation

Learned functions:

$$s^0 = h_\theta(o_{1:t}), \quad (r^k, s^k) = g_\theta(s^{k-1}, a^k), \quad (p^k, v^k) = f_\theta(s^k).$$

A simplified unroll loss is

$$\mathcal{L}_t(\theta) = \sum_{k=0}^K [\ell_v(v_t^k, z_{t+k}) + \ell_p(p_t^k, \pi_{t+k}^{\text{MCTS}}) + \mathbf{1}_{k>0} \ell_r(r_t^k, u_{t+k})] + c \|\theta\|^2.$$

Here z is an n -step bootstrapped value target, u observed reward, and π^{MCTS} normalized visit counts. A PUCT-style selection score is

$$Q(s, a) + c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}.$$

Search improves the policy target beyond the raw network prior.

7. Practical Implementation

```

import math

def select_puct(children, c_puct=1.5):
    """children: list of dicts with prior, visits, value_sum."""
    total_visits = sum(child["visits"] for child in children)
    best_score, best_index = float("-inf"), None
    for i, child in enumerate(children):
        q = child["value_sum"] / child["visits"] if child["visits"] else 0.0
        exploration = (
            c_puct * child["prior"] * math.sqrt(total_visits + 1)
            / (1 + child["visits"])
        )
        score = q + exploration
        if score > best_score:
            best_score, best_index = score, i
    return best_index

children = [
    {"prior": 0.7, "visits": 10, "value_sum": 6.0},
    {"prior": 0.3, "visits": 2, "value_sum": 1.6},
]
print(select_puct(children))

```

8. Code Explanation

PUCT combines mean backed-up value with an exploration term. High policy prior and low visit count increase exploration; repeated visits shrink the bonus. A complete MCTS also expands nodes via learned dynamics, predicts leaf value/prior, handles reward discounting, and backs values up the selected path.

9. Training / Evaluation

Replay stores observation histories, actions, rewards, and search statistics. Targets use actual rewards, n-step value bootstrap, and MCTS visit distributions. Track episodic return/win rate, network-versus-search policy strength, value calibration, reward prediction, search simulations per move, replay age, and ablations without search/model components. Use held-out levels and many seeds for generalization.

10. Complexity and Cost

Per real decision, MCTS costs roughly $O(MD)$ learned-model operations for M simulations and average depth D , though batching and tree reuse help. Training unrolls K model steps per replay sample. Large systems require substantial accelerators, replay storage, and actor/learner infrastructure. Inference is far costlier than a single policy forward pass.

11. Common Use Cases

Board games, Atari-style games, planning-rich discrete tasks, self-play systems, and research on latent world models and search-based decision-making.

12. Common Mistakes

- Saying MuZero reconstructs/predicts future observations as its core target
- Treating learned latent states as guaranteed physical states
- Forgetting reward prediction in the dynamics unroll
- Training policy only on played action instead of search visit distribution
- Comparing compute-heavy search against single-pass policies without compute accounting
- Backing up values with the wrong player perspective in zero-sum games

13. Edge Cases / Limitations

Search is expensive and difficult in large continuous action spaces. Learned latent dynamics can be inaccurate far down the tree. Legal-action structure may need environment knowledge or masking. Stochastic environments require extensions. Engineering complexity is much greater than PPO or DQN.

14. Variations

AlphaZero uses known game rules and MCTS with policy/value networks. MuZero learns latent dynamics and reward. EfficientZero improves data efficiency. Sampled MuZero handles large action spaces by sampling candidates. Stochastic MuZero models chance. Gumbel MuZero changes root/action selection and policy-improvement machinery. Dreamer-style methods also use latent models but typically optimize imagined actor-critic trajectories instead of MCTS.

15. Related Topics

Model-based RL supplies the learned dynamics idea. **MCTS** supplies planning and policy improvement. **DQN** learns Q without explicit search. **AlphaZero** is the closest known-dynamics predecessor. **World models** learn latent predictive state.

16. Interview Questions

1. **What are MuZero's three learned functions?** Representation, dynamics, and prediction.
2. **What does the dynamics model output?** Next latent state and predicted reward.
3. **Does MuZero predict raw future observations?** Not necessarily; it learns task-relevant latent predictions.
4. **Why use MCTS?** To combine lookahead, model predictions, value estimates, and exploration.
5. **What trains the policy head?** The MCTS root visit-count distribution.
6. **What trains the value head?** A bootstrapped return/outcome target.
7. **MuZero versus AlphaZero?** AlphaZero is given true transition rules; MuZero learns latent dynamics and rewards.
8. **What is PUCT?** A tree-selection score combining mean value with policy-prior exploration.
9. **Why unroll during training?** To make latent dynamics useful across multiple imagined action steps.
10. **Main practical limitation?** Heavy search/training compute and engineering complexity.

17. Practice Tasks

- Implement PUCT node selection.
- Build MCTS with a known Tic-Tac-Toe model.
- Replace rules with a learned latent transition model.
- Check value backup signs for alternating players.

- Ablate simulation count and measure strength/latency.

18. Project Ideas

Project	What it does	Tech stack	Dataset/environment	Resume value
Mini AlphaZero	Learns board-game policy through self-play	PyTorch, Python MCTS	Connect Four/Othello	Strong search + DL project
Mini MuZero	Learns latent model and search	PyTorch, Gymnasium	CartPole discretized/GridWorld	Research internship standout
Search-cost study	Evaluates strength versus simulations	PyTorch, Plotly	Board-game checkpoints	Systems/research evaluation

19. Quick Revision

- **Key idea:** learn a planning-relevant latent world model and improve actions with MCTS.
- **Main formula:** h encodes, g transitions/rewards, f predicts policy/value.
- **When to use:** discrete tasks where lookahead is valuable and compute is available.
- **Metrics:** return/win rate, search gain, prediction losses, latency.
- **Common trap:** claiming it reconstructs future observations.
- **Interview one-liner:** "MuZero learns only the model needed for value-and-policy-guided search."

20. Final Cheat Sheet

Item	Summary
Definition	Learned latent dynamics plus MCTS policy improvement
Input/output	Observation history → searched action distribution
Main steps	Encode, search/select-expand-backup, act, replay unroll
Hyperparameters	Simulations, c_{puct} , unroll length, n-step, discount
Metrics	Return/win rate, search gain, latency, model losses
Pros/cons	Powerful planning without known rules / very compute-heavy
Best uses	Planning-rich games and discrete research benchmarks

Cross-Topic Interview Map

If the interviewer asks...	Connect these ideas
"Why does RL need the Markov assumption?"	State sufficiency → Bellman recursion → TD targets
"Q-learning or SARSA?"	Off-policy greedy target versus on-policy behavior target

If the interviewer asks...	Connect these ideas
"DQN or PPO?"	Discrete off-policy replay versus general on-policy policy optimization
"Why advantage?"	$Q - V$ baseline $\rightarrow$ lower-variance policy gradient $\rightarrow$ GAE
"A2C or A3C?"	Synchronous batches versus asynchronous shared updates
"Model-free or model-based?"	Interaction efficiency versus model bias/planning compute
"What breaks offline?"	Missing action support $\rightarrow$ extrapolation error $\rightarrow$ conservative learning/OPE
"What makes RLHF difficult?"	Learned proxy reward $\rightarrow$ distribution shift/reward hacking $\rightarrow$ KL and human evaluation
"What makes MuZero special?"	Learned latent dynamics trained for reward/value/policy, used inside MCTS

End-to-End Revision Checklist

- Can define the agent-environment loop and distinguish state from observation.
- Can derive return, V^π , Q^π , A^π , and their Bellman equations.
- Can write Q-learning, SARSA, DQN, policy-gradient, GAE, and PPO losses from memory.
- Can explain terminal versus time-limit truncation in bootstrapped targets.
- Can choose algorithms for discrete/continuous actions and online/offline data.
- Can discuss reward hacking, exploration safety, multi-agent non-stationarity, and model bias.
- Can describe RLHF and MuZero as systems, including their evaluation and compute costs.